المدرسة العليا الخاصة للعلوم التطبيقية و إدارة الأعمال
ECOLE SUPÉRIEURE PRIVÉE DES SCIENCES APPLIQUÉES ET DE MANAGEMENT

RAPPORT DE STAGE DE PROJET DE FIN D'ETUDES

Intitulé du stage

Conception et Développement d'une Plateforme Web de Gestion du Processus de Recrutement — JOB4YOU

Réalisé par : Haythem Bargaoui

Entreprise d'accueil

Sopra HR Software Tunisie

Encadrant Entreprise
M. Mones Ben Mabrouk

Encadrant SESAME
M. Malek Bouzaiene

Année Universitaire : 2025 - 2026

Dédicace

Je dédie ce travail à mes parents, dont l'amour inconditionnel, le soutien de chaque instant et les sacrifices ont rendu ce parcours possible. Aucun mot ne pourrait vraiment rendre justice à ma gratitude envers eux.

Mes frères et sœurs m'ont accompagné par leurs encouragements et leur présence, y compris dans les moments les plus difficiles.

Mes amis et collègues ont eux aussi eu leur part, par leur soutien moral et les bons moments partagés pendant cette expérience.

Je n'oublie enfin personne d'autre ayant, de près ou de loin, contribué à la réussite de ce travail.

Remerciements

Au terme de ce projet de fin d'études, j'exprime ma sincère reconnaissance à toutes les personnes qui, de près ou de loin, ont contribué à ce travail.

Mon encadrant entreprise, M. Mones Ben Mabrouk, m'a accompagné avec rigueur et disponibilité tout au long de ce stage ; ses conseils avisés et la confiance qu'il m'a accordée ont compté autant que ses retours techniques, déterminants dans la réussite de ce projet.

Du côté académique, M. Malek Bouzaiene a suivi mon travail avec attention, toujours disponible pour des conseils précieux qui ont orienté aussi bien la rédaction de ce rapport que la démarche méthodologique suivie.

J'ai également pu compter sur un accueil chaleureux de la part de toute l'équipe de Sopra HR Software Tunisie, dont l'esprit d'équipe et le partage d'expertise m'ont permis d'évoluer dans un environnement professionnel stimulant et formateur.

Je n'oublie pas non plus le corps professoral et administratif de SESAME, pour la qualité de la formation reçue tout au long de mon cursus d'ingénieur.

Enfin, je remercie les membres du jury pour l'honneur qu'ils me font en acceptant d'évaluer ce travail.

Résumé

Mené en alternance chez Sopra HR Software Tunisie sur sept mois, ce Projet de Fin d'Études consistait à concevoir et développer JOB4YOU, une plateforme web destinée à centraliser et automatiser le cycle de recrutement interne de l'entreprise, de la publication des offres d'emploi jusqu'à la décision finale, en passant par le suivi des candidatures, la planification des entretiens et la collecte des feedbacks des managers.

J'ai construit la solution autour d'une architecture trois-tiers découplée : un backend Java/Spring Boot expose des API REST sécurisées par JWT et gère finement les rôles (Administrateur, RH, Manager, Candidat), un frontend Angular assure une interface moderne et responsive, et PostgreSQL sert de base de données relationnelle. Des agents d'automatisation n8n, couplés à un service de scoring par intelligence artificielle (API Cohere), prennent en charge l'analyse automatique des CV, en complément d'un système de notifications par e-mail à chaque étape clé du processus.

Ce rapport présente successivement le cadre général du projet, l'analyse et la spécification des besoins, la conception de la solution, sa réalisation, puis un bilan de cette expérience et les perspectives d'évolution de la plateforme.

Mots-clés : recrutement, plateforme web, Spring Boot, Angular, PostgreSQL, API REST, JWT, intelligence artificielle, automatisation, n8n.

Abstract

During my seven-month work-study End-of-Studies Project at Sopra HR Software Tunisia, I designed and built JOB4YOU, a web platform for managing the company's internal recruitment process end to end — from publishing job offers through application tracking, interview scheduling, manager feedback collection, and the final hiring decision.

The solution rests on a decoupled three-tier architecture: a Java/Spring Boot backend exposes REST APIs secured by JWT authentication with fine-grained role management (Admin, HR, Manager, Candidate), an Angular front-end delivers a modern, responsive interface, and PostgreSQL handles relational storage. n8n automation agents, paired with an AI-based scoring service built on the Cohere API, handle automatic resume analysis, alongside an email notification system triggered at each key step.

The report walks through the general project framework, requirements analysis and specification, the solution's design and implementation, before closing with an assessment of the experience and the platform's future development perspectives.

Keywords: recruitment, web platform, Spring Boot, Angular, PostgreSQL, REST API, JWT, artificial intelligence, automation, n8n.

Table des matières

Table des matières 6
Liste des figures 9
Liste des tableaux 10
Introduction Générale 11
Chapitre 1 : Cadre Général du Projet 12
1.1 Introduction 12
1.2 Présentation de l'organisme d'accueil 12
1.2.1 Le groupe Sopra Steria 12
1.2.2 Sopra HR Software 12
1.3 Étude de l'existant 12
1.4 Critique de l'existant et problématique 13
1.5 Objectifs du projet 13
1.6 Présentation de la solution proposée : JOB4YOU 13
1.7 Méthodologie de travail adoptée 14
1.8 Planification du projet 14
1.9 Conclusion 15
Chapitre 2 : Analyse et Spécification des Besoins 16
2.1 Introduction 16
2.2 Étude fonctionnelle 16
2.2.1 Identification des acteurs du système 16
2.2.2 Identification des besoins fonctionnels 17
2.2.3 Identification des besoins non fonctionnels 18
2.2.4 Diagramme de cas d'utilisation 18
2.2.5 Description textuelle de quelques cas d'utilisation 19
2.2.6 Diagramme de séquence système 20
2.3 Étude technique 20
2.3.1 Architecture physique 20

2.3.2 Côté client — Angular 21
2.3.3 Côté serveur — Spring Boot 21
2.3.4 La sécurité — Spring Security et JWT 21
2.4 Conclusion 21
Chapitre 3 : Conception 22
3.1 Introduction 22
3.2 Diagramme des paquetages 22
3.3 Diagramme de classes global 22
3.4 Diagrammes de séquence objet 24
3.4.1 Planifier un entretien 24
3.4.2 Décision finale de recrutement 25
3.4.3 Analyse IA du CV (extraction, scoring via n8n et Cohere) 25
3.4.4 Passage en évaluation technique — routage et notification des managers 26
3.5 Diagramme d'états — Cycle de vie d'une candidature 27
3.6 Conception de la base de données 28
3.7 Diagramme de déploiement 30
3.8 Diagramme de composants 30
3.9 Diagramme d'activité — Vue d'ensemble du processus de recrutement 31
3.10 Conclusion 32
Chapitre 4 : Réalisation 34
4.1 Introduction 34
4.2 Environnement de travail 34
4.2.1 Environnement matériel 34
4.2.2 Environnement logiciel 34
4.2.3 Autres 35
4.2.4 Sécurité applicative 35
4.3 Interfaces graphiques de l'application 35
4.4 Tests 43
4.4.1 Tests unitaires (JUnit 5 / Mockito) 43

4.4.2 Tests fonctionnels manuels 44
4.4.3 Tests des API avec Postman et Swagger 45
4.4.4 Vérification de bout en bout 46
4.5 Difficultés rencontrées et solutions apportées 46
4.5.1 Notification incomplète des managers 46
4.5.2 Distinguer un rejet RH d'un rejet manager 46
4.5.3 Cohérence des transitions de statut 46
4.5.4 Filtrage incomplet des routes de lecture pour les managers 47
4.5.5 Solliciter le manager au bon moment du processus 47
4.5.6 Pertinence du matching Job idéal 47
4.5.7 Traitement distinct des candidatures retirées 47
4.5.8 Entretien téléphonique RH obligatoire avant acceptation 48
4.5.9 Découplage entre planification d'entretien et statut du candidat 48
4.5.10 Fiabiliser l'intégration Google Meet et le format des webhooks n8n 48
4.5.11 Cloisonnement des données dans le chatbot candidat 48
4.6 Protection des données et éthique de l'IA 49
4.7 Bilan des fonctionnalités réalisées 50
4.8 Conclusion 51
Conclusion Générale et Perspectives 52
Bibliographie 53
Glossaire 54

Liste des figures

Figure 2.1 : Diagramme de cas d'utilisation global 19
Figure 2.2 : Diagramme de séquence système — Soumettre une candidature 20
Figure 2.3 : Architecture physique de la plateforme JOB4YOU 21
Figure 3.1 : Diagramme des paquetages 22
Figure 3.2 : Diagramme de classes global 23
Figure 3.3 : Diagramme de séquence objet — Planifier un entretien 24
Figure 3.4 : Diagramme de séquence objet — Décision finale de recrutement 25
Figure 3.5 : Diagramme de séquence objet — Analyse IA du CV 26
Figure 3.6 : Diagramme de séquence objet — Routage et notification des managers 26
Figure 3.7 : Diagramme d'états du cycle de vie d'une candidature 27
Figure 3.8 : Schéma relationnel de la base de données PostgreSQL 29
Figure 3.9 : Diagramme de déploiement de la plateforme JOB4YOU 30
Figure 3.10 : Diagramme de composants de la plateforme JOB4YOU 31
Figure 3.11 : Diagramme d'activité — Vue d'ensemble du processus de recrutement 32
Figure 4.1 : Page d'accueil publique — recherche d'offres d'emploi 35
Figure 4.2 : Page de connexion 36
Figure 4.3 : Gestion des offres d'emploi (espace RH) 36
Figure 4.4 : Gestion des candidats — vue liste avec score IA 37
Figure 4.5 : Modification d'un dossier candidat 37
Figure 4.6 : Tableau Kanban du pipeline de recrutement 38
Figure 4.7 : Gestion des entretiens 38
Figure 4.8 : Gestion des feedbacks 39
Figure 4.9 : Gestion des utilisateurs et des rôles (espace Administrateur) 39
Figure 4.10 : Paramétrage du scoring IA par famille de métier 40
Figure 4.11 : Journal d'audit 40
Figure 4.12 : Assistant conversationnel candidat (chatbot) 41
Figure 4.13 : Soumission d'une candidature avec dépôt de CV 41
Figure 4.14 : Fonctionnalité « Job idéal » — suggestions d'offres selon le CV 42
Figure 4.15 : Analyse détaillée d'une candidature par intelligence artificielle 42
Figure 4.16 : Espace Manager — dossiers validés par le RH 43

Liste des tableaux

Tableau 2.1 : Matrice des responsabilités par rôle 16

Tableau 4.1 : Synthèse des tests fonctionnels réalisés 44

Tableau 4.2 : Bilan des fonctionnalités réalisées 50

Introduction Générale

La fonction Ressources Humaines prend une place de plus en plus stratégique dans les entreprises, portée par la digitalisation accélérée des processus métiers. Le recrutement, première étape du cycle de vie du collaborateur, conditionne directement la qualité du capital humain d'une organisation — et pourtant, dans beaucoup de structures, ce processus reste fragmenté entre des outils hétérogènes : messagerie électronique, tableurs, formulaires papier. Cette dispersion nuit à la traçabilité des décisions, à la collaboration entre intervenants, et à l'expérience vécue par les candidats.

C'est dans ce contexte que Sopra HR Software Tunisie m'a confié le projet JOB4YOU, dans le cadre de mon Projet de Fin d'Études en alternance : concevoir et développer une plateforme web centralisée, sécurisée et intuitive, couvrant l'intégralité du processus de recrutement interne, de la publication d'une offre jusqu'à la décision finale d'embauche, en passant par la gestion des candidatures, l'organisation des entretiens et la collecte des retours des managers.

Ce rapport retrace la démarche suivie tout au long du projet, en quatre chapitres. Le premier pose le cadre général — organisme d'accueil, étude de l'existant, problématique et méthodologie adoptée. Le deuxième analyse et spécifie les besoins fonctionnels et non fonctionnels de la plateforme. Le troisième détaille la conception de la solution, sur le plan architectural comme sur celui des modèles de données. Le quatrième décrit la réalisation technique : environnement de travail et principales fonctionnalités développées. Le rapport se conclut par un bilan du travail effectué et les perspectives d'évolution envisagées pour la plateforme.

Chapitre 1 : Cadre Général du Projet

J'ai rejoint Sopra HR Software Tunisie pour sept mois d'alternance, avec pour mission de digitaliser un processus de recrutement resté, jusque-là, largement manuel. Ce chapitre revient sur le point de départ de ce travail : l'entreprise qui m'a accueilli, les lacunes que j'ai concrètement constatées dans l'existant, la problématique qui en découle, et la manière dont j'ai organisé mon travail sur toute la durée du stage.

1.1 Introduction

Je situe d'abord le cadre organisationnel du projet JOB4YOU, avant de détailler la démarche qui m'a mené à la solution finalement retenue.

1.2 Présentation de l'organisme d'accueil

1.2.1 Le groupe Sopra Steria

Sopra Steria est un groupe européen de conseil, de services numériques et d'édition de logiciels, reconnu pour son expertise sectorielle et son approche de bout en bout sur les projets de transformation numérique. Né du rapprochement entre Sopra Group et Steria en 2014, il rassemble plusieurs dizaines de milliers de collaborateurs répartis dans de nombreux pays, et intervient auprès de grands comptes et d'administrations publiques dans des secteurs variés — finance, télécommunications, énergie, secteur public, ou encore ressources humaines.

1.2.2 Sopra HR Software

Sopra HR Software est la filiale du groupe spécialisée dans l'édition de solutions logicielles pour la gestion des ressources humaines : paie, gestion administrative du personnel, gestion des talents et gestion du recrutement. En Tunisie, l'entité constitue un centre de développement et de compétences qui contribue à la conception, au développement et à la maintenance des solutions du groupe, tout en gérant en interne ses propres processus, dont le recrutement.

C'est précisément pour répondre à ce besoin interne de digitalisation que le projet JOB4YOU m'a été proposé.

1.3 Étude de l'existant

Avant la mise en place du projet, le processus de recrutement au sein de Sopra HR Software reposait sur une combinaison d'outils disparates et de tâches manuelles :

- La publication des offres d'emploi et leur diffusion s'effectuaient de manière peu centralisée, sans visibilité unifiée sur l'ensemble des postes ouverts.
- La réception des candidatures et des CV se faisait principalement par messagerie électronique, rendant leur classement, leur archivage et leur consultation fastidieux.
- Le suivi de l'avancement d'une candidature (CV reçu, examiné, entretien planifié, décision) n'était pas formalisé dans un outil unique, ce qui entraînait une perte d'information entre les différents intervenants (RH, managers).
- La planification des entretiens et la collecte des feedbacks des managers se faisaient de manière informelle, sans traçabilité ni historique structuré.

- Aucune notification automatique n'informait les candidats de l'état d'avancement de leur candidature, ce qui pouvait nuire à leur expérience et à l'image de l'entreprise.
- L'évaluation initiale des candidatures reposait entièrement sur l'analyse manuelle des CV par les équipes RH, un processus chronophage à mesure que le volume de candidatures augmente.

1.4 Critique de l'existant et problématique

Cette analyse de l'existant fait ressortir plusieurs limites majeures. Sans outil centralisé, l'information se disperse entre plusieurs canaux — e-mails, fichiers, échanges oraux — ce qui complique la collaboration entre responsables RH et managers impliqués dans la décision de recrutement. Le manque de traçabilité des décisions et des statuts rend le suivi de chaque candidature difficile et allonge les délais de traitement. L'absence d'automatisation, enfin, que ce soit dans la présélection des candidats ou dans la communication avec eux, freine l'efficacité et la scalabilité du processus, surtout à mesure que le volume de candidatures grandit.

De ces constats découle la problématique suivante : comment centraliser, sécuriser et automatiser l'ensemble du processus de recrutement — de la publication d'une offre jusqu'à la décision finale — tout en offrant une visibilité en temps réel à tous les acteurs concernés (administrateurs, RH, managers et candidats) ?

1.5 Objectifs du projet

Pour répondre à cette problématique, les objectifs suivants ont été fixés pour ce Projet de Fin d'Études :

- Concevoir et développer une plateforme web unique regroupant l'ensemble des étapes du processus de recrutement.
- Mettre en place une gestion des rôles et des accès (Administrateur, RH, Manager, Candidat) garantissant que chaque acteur dispose des fonctionnalités et de la visibilité correspondant à son périmètre.
- Sécuriser les échanges et les accès à la plateforme via un mécanisme d'authentification robuste (JSON Web Token).
- Automatiser la pré-analyse des candidatures grâce à un système de scoring assisté par intelligence artificielle, afin d'accélérer le tri initial des CV tout en laissant la décision finale aux équipes RH et managers.
- Intégrer un système de notifications automatiques par e-mail informant les candidats et les parties prenantes internes à chaque étape clé du processus.
- Fournir aux équipes RH des outils de pilotage (tableau Kanban, tableau de bord, indicateurs) permettant un suivi visuel et en temps réel de l'avancement des recrutements.

1.6 Présentation de la solution proposée : JOB4YOU

La solution proposée, baptisée JOB4YOU, est une plateforme web de gestion du processus de recrutement structurée autour de quatre profils d'utilisateurs :

- Le Candidat, qui peut consulter les offres d'emploi publiées, soumettre une candidature accompagnée de son CV, et suivre l'état d'avancement de son dossier.
- Le Responsable RH, qui publie et gère les offres d'emploi, examine les candidatures reçues, valide les dossiers, planifie les entretiens et communique les décisions finales aux candidats.
- Le Manager, qui intervient dans le processus de sélection après validation RH, mène ou participe aux entretiens et fournit un feedback structuré sur les candidats évalués.
- L'Administrateur, qui supervise la plateforme dans son ensemble, gère les comptes utilisateurs, les rôles et les paramètres globaux du système, notamment ceux liés au scoring par intelligence artificielle.

Techniquement, la plateforme s'appuie sur une architecture découplée : un backend Spring Boot expose des API REST sécurisées, un frontend Angular porte l'interface utilisateur, PostgreSQL sert de base de données relationnelle, et des agents d'automatisation n8n prennent en charge l'analyse des CV par intelligence artificielle (via l'API Cohere) ainsi que l'envoi des notifications aux différents acteurs. Le chapitre consacré à la conception détaille cette architecture.

1.7 Méthodologie de travail adoptée

Les besoins évoluant au fil du stage, et les retours réguliers de mon encadrant entreprise étant essentiels, j'ai adopté une démarche itérative et incrémentale, inspirée des principes Agile/Scrum. Le développement s'est ainsi organisé en cycles courts : chaque fonctionnalité — gestion des candidats, des entretiens, des feedbacks, scoring IA, notifications — était analysée, conçue, développée puis validée avant de passer à la suivante.

Des points de synchronisation réguliers avec mon encadrant m'ont permis d'ajuster les priorités et d'affiner les spécifications au fil de l'avancement. Git/GitHub a servi tout du long à assurer la traçabilité du code, faciliter les retours en arrière et documenter l'évolution du projet à travers l'historique des commits.

1.8 Planification du projet

Le projet s'est étalé sur sept mois, du 02 février au 31 août 2025, découpés en plusieurs grandes phases que résume le tableau suivant.

Phase	Période indicative	Activités principales
Prise en main et cadrage	Février 2025	Intégration au sein de l'équipe, étude de l'existant, analyse du besoin, rédaction du cahier des charges
Analyse et conception	Mars 2025	Spécification des besoins, modélisation UML [8] (cas d'utilisation, séquence, classes), conception de la base de données
Développement — Socle applicatif	Avril - Mai 2025	Mise en place de l'architecture backend/frontend, authentification JWT, gestion des rôles, gestion des offres et des candidatures
Développement — Fonctionnalités avancées	Juin - Juillet 2025	Gestion des entretiens et des feedbacks, scoring IA, agents n8n,

Phase	Période indicative	Activités principales
		notifications e-mail, tableau Kanban
Tests, corrections et finalisation	Août 2025	Tests fonctionnels, corrections de bugs, améliorations UX et sécurité, rédaction du rapport

1.9 Conclusion

Les limites du processus existant chez Sopra HR Software sont maintenant posées, tout comme la méthodologie suivie pendant le stage. Reste à entrer dans le vif du sujet : quels besoins précis JOB4YOU devait-il couvrir ? C'est l'objet du chapitre suivant.

Chapitre 2 : Analyse et Spécification des Besoins

Le contexte posé, restait à traduire la problématique en besoins concrets : qui utilise la plateforme, pour faire quoi, et avec quelles contraintes ? C'est ce que je détaille dans ce chapitre, acteur par acteur, avant de justifier les choix techniques préliminaires qui ont guidé l'architecture retenue.

2.1 Introduction

Recueillir les besoins n'a rien eu d'un exercice ponctuel : cela s'est étalé sur les premières semaines du stage, au fil des échanges avec mon encadrant et de l'observation directe du processus RH. Ce qui suit est le résultat de cette phase, et sert de base directe à la conception présentée au chapitre suivant.

2.2 Étude fonctionnelle

2.2.1 Identification des acteurs du système

Quatre acteurs interviennent sur la plateforme JOB4YOU, chacun avec ses propres privilèges et fonctionnalités :

- Le Candidat : utilisateur externe, non authentifié ou disposant d'un compte, qui consulte les offres d'emploi publiées et soumet des candidatures.
- Le Responsable RH : acteur central du processus, il publie les offres, examine et valide les candidatures, planifie les entretiens et communique les décisions.
- Le Manager : intervient une fois le dossier validé par le RH ; il consulte les candidatures relevant de son périmètre, mène ou participe aux entretiens, fournit un feedback structuré et peut prendre la décision finale (acceptation ou refus) sur les dossiers de son périmètre.
- L'Administrateur : acteur transverse chargé de la gestion des comptes utilisateurs, des rôles, du paramétrage du scoring IA par famille de métier et de la supervision globale de la plateforme (journal d'audit).

Le tableau ci-dessous précise, action par action, les rôles habilités — une répartition qui correspond exactement aux règles d'autorisation implémentées côté backend.

Tableau 2.1 : Matrice des responsabilités par rôle

Action	RH	Manager	Administrateur	Candidat
Publier / gérer une offre d'emploi	Oui	Non	Oui	Non
Valider un CV (passage CV_REVIEWED)	Oui	Non	Oui	Non
Planifier un entretien	Oui	Oui (son périmètre)	Oui	Non
Saisir un feedback	Non	Oui	Non	Non
Décision finale (acceptation / refus)	Oui	Oui (son périmètre)	Oui	Non

Action	RH	Manager	Administrateur	Candidat
Gérer les utilisateurs et les rôles	Non	Non	Oui	Non
Paramétrer le scoring IA	Non	Non	Oui	Non
Consulter le journal d'audit	Oui	Non	Oui	Non

J'ai volontairement donné au RH et à l'Administrateur un droit de décision qui recouvre celui du Manager : un RH peut ainsi corriger ou débloquer un dossier en l'absence du manager concerné, tout en gardant une trace, via le journal d'audit, de qui a réellement pris chaque décision.

2.2.2 Identification des besoins fonctionnels

Les échanges avec mon encadrant entreprise et l'étude de l'existant ont fait émerger les besoins fonctionnels suivants :

- Authentification et gestion des sessions sécurisée par jeton JWT, avec gestion différenciée des accès selon le rôle de l'utilisateur.
- Gestion des offres d'emploi : création, modification, publication, clôture et expiration automatique des offres dont la date limite est dépassée.
- Gestion des candidatures : soumission en ligne avec dépôt de CV, consultation, filtrage et suivi de l'état d'avancement de chaque dossier.
- Analyse automatique des candidatures par intelligence artificielle : extraction du contenu du CV et calcul d'un score de pertinence (technique, communication, adéquation séniorité) avec possibilité de correction manuelle par le RH.
- Gestion des entretiens : planification, changement de statut (planifié, en cours, terminé, annulé, reprogrammé), envoi d'invitations et de rappels automatiques.
- Gestion des feedbacks : saisie d'un retour structuré par le manager après entretien, validation par le RH et notification au candidat.
- Prise de décision finale : validation ou refus d'un candidat par le manager ou par le RH, avec garde-fou empêchant toute décision sur un dossier non encore validé par le RH.
- Notifications automatiques par e-mail à chaque étape clé du processus (confirmation de candidature, invitation à un entretien, décision finale).
- Tableau de pilotage visuel de type Kanban permettant de visualiser en un coup d'œil la répartition des candidatures par statut.
- Gestion des utilisateurs, des rôles et paramétrage des profils de pondération du score IA par famille de métier.

- Matching inverse (« Job idéal ») : un visiteur peut déposer son CV sans postuler à une offre précise ; le système le compare aux offres actives et lui suggère les plus pertinentes avec un score de compatibilité.

2.2.3 Identification des besoins non fonctionnels

Au-delà des fonctionnalités métier, la plateforme doit aussi répondre aux exigences non fonctionnelles suivantes :

- Sécurité : authentification robuste par JWT, hachage des mots de passe, contrôle d'accès basé sur les rôles (RBAC) à la fois côté frontend et côté backend.
- Performance : temps de réponse adapté à une utilisation interne quotidienne, chargement asynchrone des traitements longs (scoring IA, envoi d'e-mails) afin de ne pas bloquer l'utilisateur.
- Ergonomie et responsive design : interface claire, intuitive et utilisable aussi bien sur poste de travail que sur tablette.
- Fiabilité : garde-fous métier empêchant les transitions de statut incohérentes (ex. décision finale sur un dossier non validé par le RH, entretien planifié sur un dossier clos).
- Maintenabilité et évolutivité : architecture modulaire en couches, séparation claire entre frontend, backend et automatisation, facilitant l'ajout de nouvelles fonctionnalités.
- Traçabilité : journalisation des actions sensibles (changements de statut, corrections de score) dans un journal d'audit.

2.2.4 Diagramme de cas d'utilisation

La figure ci-dessous donne une vue synthétique des principaux cas d'utilisation de la plateforme, regroupés par acteur.

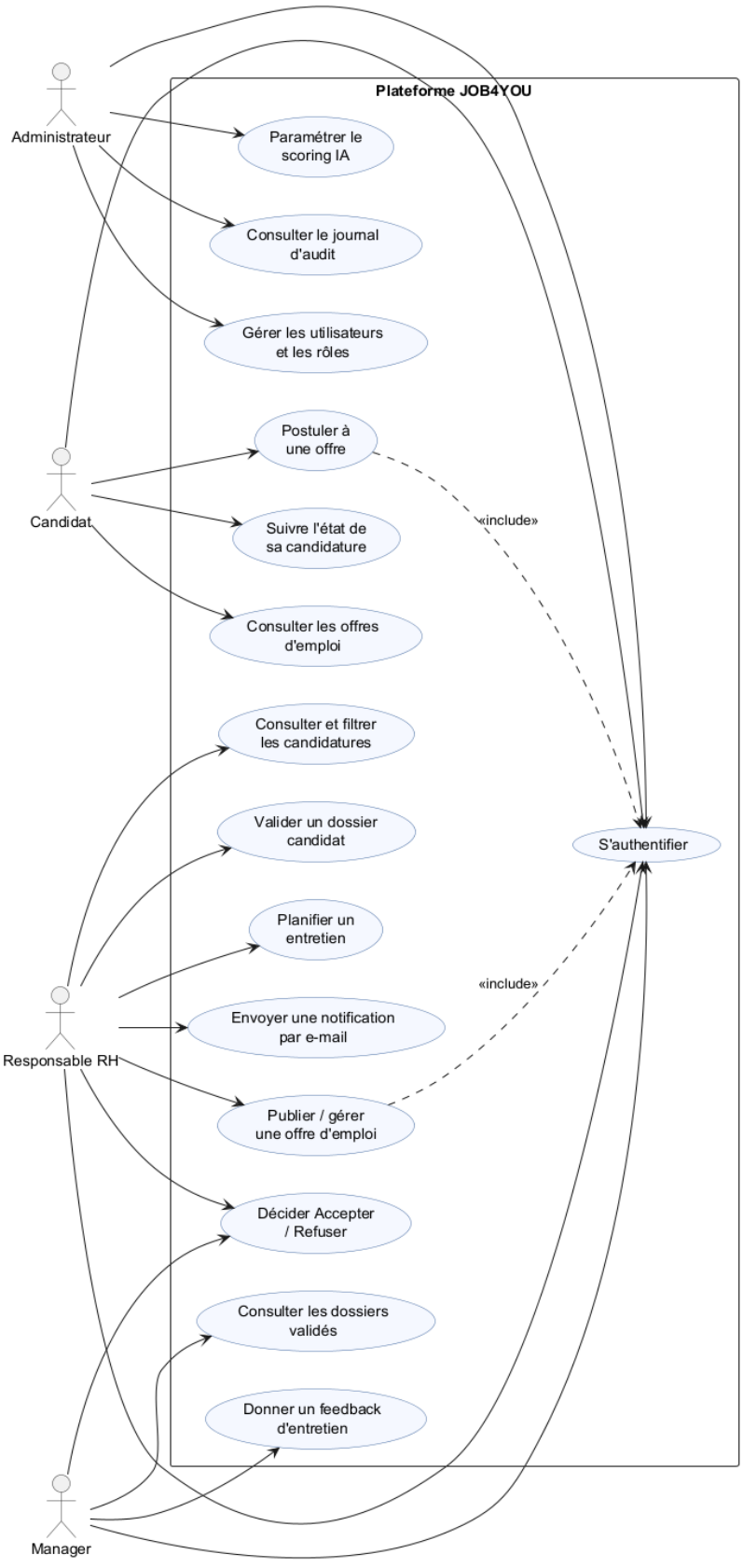


Figure 2.1 : Diagramme de cas d'utilisation global

2.2.5 Description textuelle de quelques cas d'utilisation

Le tableau qui suit détaille le scénario nominal de deux cas d'utilisation représentatifs du processus métier.

Élément	Cas d'utilisation « Postuler à une offre »	Cas d'utilisation « Décider Accepter / Refuser »
Acteur principal	Candidat	Manager
Pré-condition	L'offre est publiée (statut ACTIVE)	Le dossier a été validé par le RH (statut ≥ CV_REVIEWED)
Scénario nominal	Le candidat consulte l'offre, remplit le formulaire de candidature et dépose son CV. Le système enregistre la candidature, envoie un e-mail de confirmation et déclenche l'analyse IA du CV.	Le manager consulte le dossier du candidat, ses scores et éventuels feedbacks, puis valide une décision d'acceptation ou de refus.
Post-condition	La candidature est créée avec le statut APPLIED, puis mise à jour automatiquement selon le score IA.	Le statut du candidat passe à ACCEPTED ou MANAGER_REJECTED et un e-mail de décision est envoyé au candidat.
Exception	Candidature déjà déposée pour cette offre : le système rejette la demande.	Dossier non encore validé par le RH : le système refuse la décision et retourne un message d'erreur explicite.

2.2.6 Diagramme de séquence système

Le diagramme ci-après illustre les échanges entre le candidat, le système et l'agent d'intelligence artificielle lors du dépôt d'une candidature.

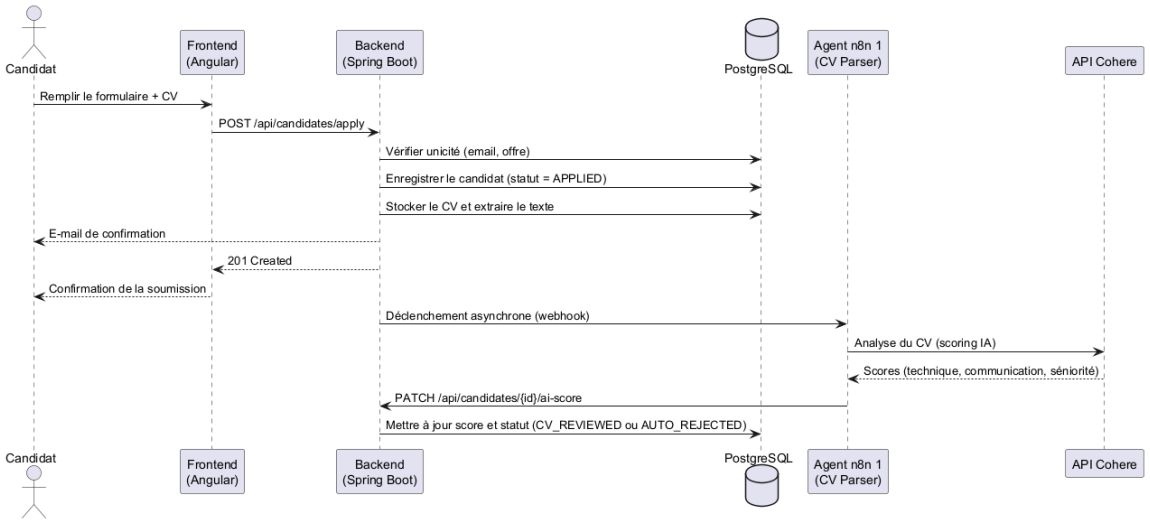


Figure 2.2 : Diagramme de séquence système — Soumettre une candidature

2.3 Étude technique

2.3.1 Architecture physique

La plateforme s'organise en architecture distribuée par couches, comme le montre la figure suivante : un frontend Angular [3], [13] consommé par le navigateur, un backend Spring Boot [1], [12] exposant des API REST sécurisées, une base de données PostgreSQL [4], et un serveur n8n [7] responsable de l'automatisation — scoring IA via l'API

Cohere [6], notifications, et une intégration réelle avec Google Calendar, qui crée l'événement et génère automatiquement le lien Google Meet correspondant.

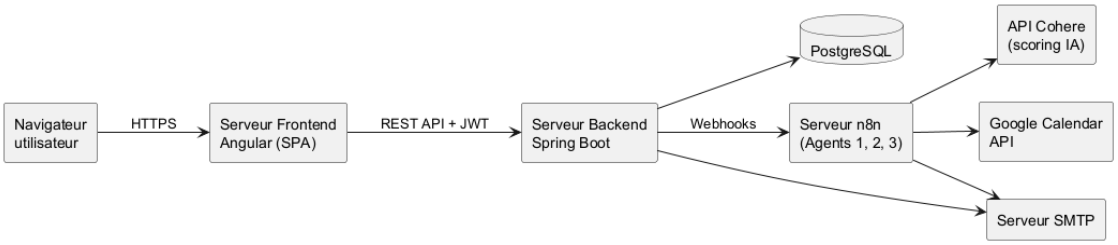


Figure 2.3 : Architecture physique de la plateforme JOB4YOU

2.3.2 Côté client — Angular

J'ai développé le frontend avec Angular, en architecture par composants autonomes (« standalone components »), organisée en pages publiques (accueil, offres d'emploi, candidature en ligne), pages d'administration (candidats, entretiens, feedbacks, offres, utilisateurs, tableau Kanban, tableau de bord) et services partagés qui assurent la communication avec l'API REST du backend. Bootstrap prend en charge la mise en forme des interfaces.

2.3.3 Côté serveur — Spring Boot

Côté serveur, j'ai développé le backend en Java avec Spring Boot 3, selon une architecture en couches classique : contrôleurs REST, services métier, repositories Spring Data JPA et entités JPA mappées sur la base PostgreSQL. Cette organisation garde les responsabilités bien séparées et facilite les évolutions futures.

2.3.4 La sécurité — Spring Security et JWT

L'accès à la plateforme repose sur Spring Security [2], couplé à une authentification par jeton JWT (JSON Web Token). À la connexion, le serveur génère un jeton signé portant l'identité et les rôles de l'utilisateur ; ce jeton est ensuite transmis dans l'en-tête de chaque requête, ce qui authentifie l'utilisateur sans nécessiter d'état de session côté serveur. Chaque point d'entrée de l'API est protégé via l'annotation @PreAuthorize, garantissant que seuls les rôles autorisés (Administrateur, RH, Manager) accèdent à une ressource donnée.

2.4 Conclusion

Acteurs, besoins et architecture technique étant désormais clairement identifiés, une question s'est imposée : comment traduire tout cela en un modèle concret, capable de guider le développement ? C'est ce que j'aborde dans le chapitre suivant, consacré à la conception.

Chapitre 3 : Conception

Passer d'une liste de besoins à un système cohérent implique des choix : comment organiser le code, comment modéliser les données, comment éviter qu'un candidat se retrouve dans un état incohérent. Ce chapitre documente les principales décisions de conception derrière JOB4YOU — paquetages, diagramme de classes, scénarios d'interaction et machine à états du cycle de vie d'une candidature.

3.1 Introduction

Chaque modèle présenté ici sert directement l'implémentation décrite au chapitre 4 : je n'ai rien modélisé pour la forme, chaque diagramme correspond à une décision réellement prise pendant le développement.

3.2 Diagramme des paquetages

La figure suivante illustre l'organisation de l'application en grands ensembles fonctionnels (paquetages), côté frontend comme côté backend, ainsi que leurs interactions avec les services externes.

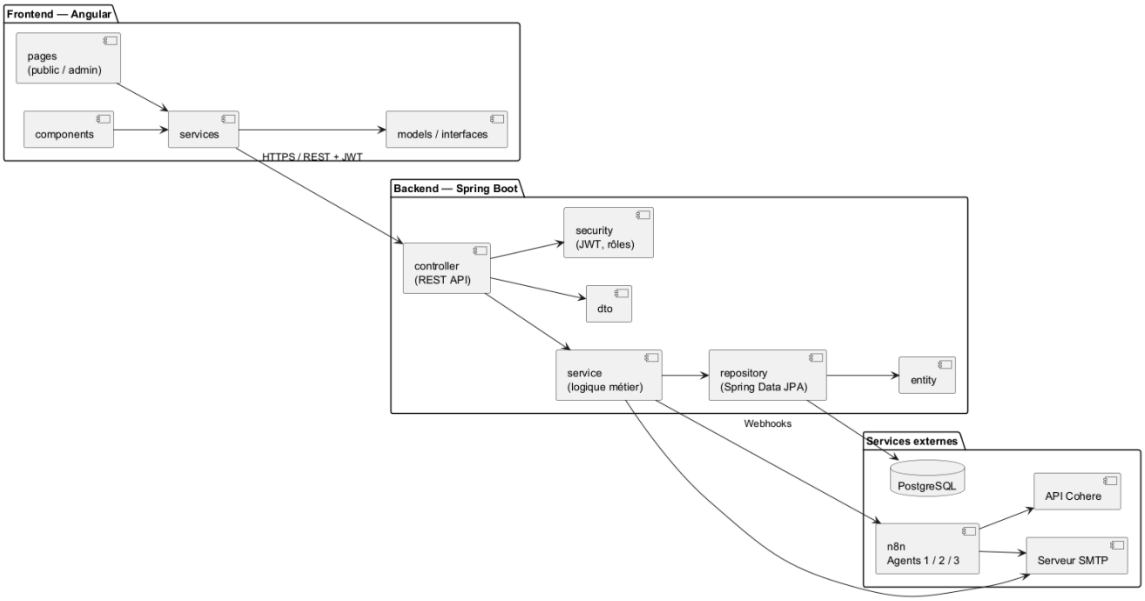


Figure 3.1 : Diagramme des paquetages

Côté backend, j'ai suivi un découpage classique en couches : la couche controller expose les API REST, la couche service porte la logique métier (règles de gestion, garde-fous, orchestration des appels vers n8n), la couche repository assure la persistance via Spring Data JPA, et les entités JPA modélisent le domaine métier. Côté frontend, chaque page consomme des services Angular dédiés à une ressource métier (candidats, entretiens, feedbacks, offres), qui communiquent eux-mêmes avec le backend via des appels HTTP authentifiés.

3.3 Diagramme de classes global

Le diagramme de classes ci-dessous présente le modèle de données central de la plateforme, ainsi que les relations entre ses principales entités métier.

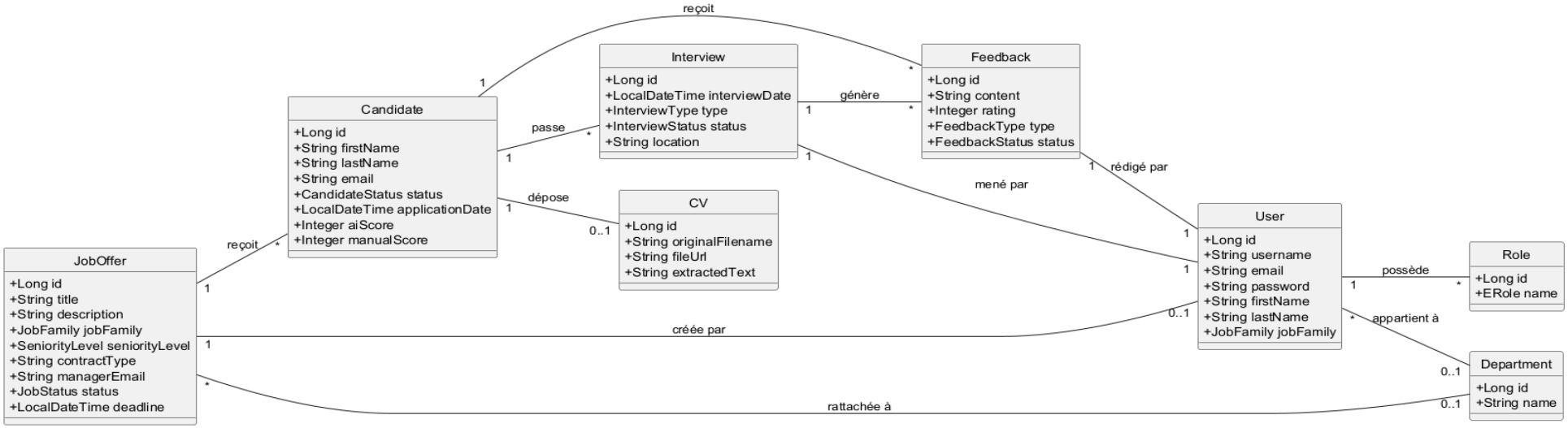


Figure 3.2 : Diagramme de classes global

Ce modèle fait ressortir plusieurs relations structurantes : un candidat dépose au plus un CV et peut passer par plusieurs entretiens, chacun pouvant donner lieu à un ou plusieurs feedbacks rédigés par un utilisateur (manager ou RH). Une offre d'emploi, rattachée à un département, reçoit plusieurs candidatures. Quant à la classe User, elle centralise l'identité des utilisateurs internes (RH, managers, administrateurs) et porte, via la classe Role, un système de rôles multiples pour une gestion fine des permissions.

3.4 Diagrammes de séquence objet

Les quatre diagrammes qui suivent détaillent, au niveau objet, deux scénarios clés du processus de recrutement : la planification d'un entretien et la décision finale de recrutement.

3.4.1 Planifier un entretien

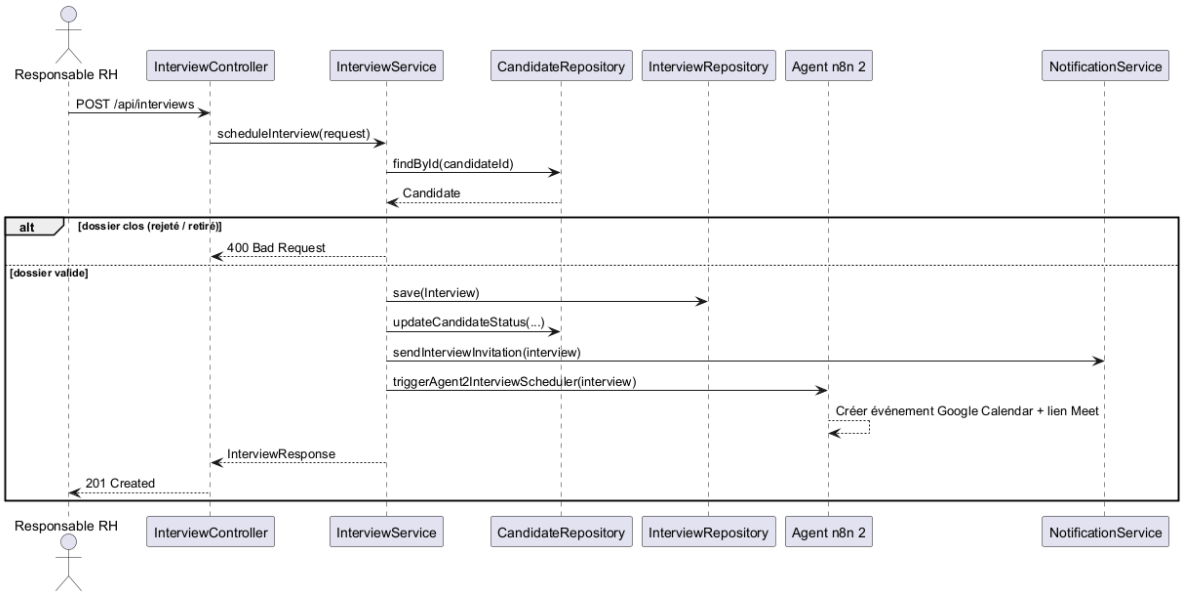


Figure 3.3 : Diagramme de séquence objet — Planifier un entretien

Ce diagramme met en avant un garde-fou métier important : avant toute planification, le service s'assure que le dossier du candidat n'est pas dans un état clos négativement (rejeté ou retiré), pour éviter de planifier un entretien sur une candidature déjà écartée.

3.4.2 Décision finale de recrutement

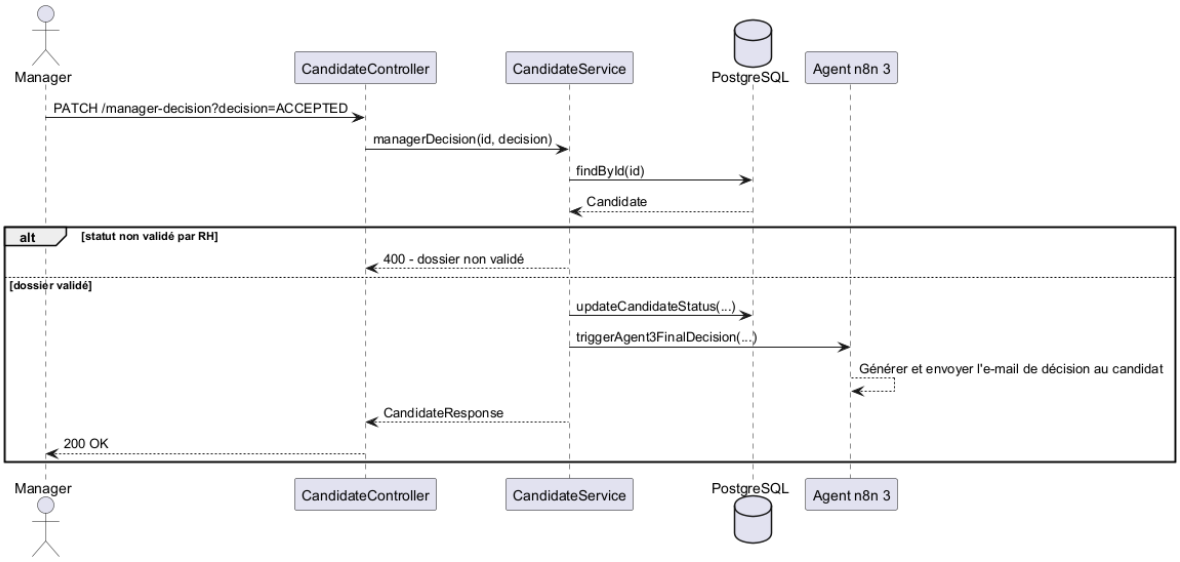


Figure 3.4 : Diagramme de séquence objet — Décision finale de recrutement

Ce second diagramme illustre un autre garde-fou central de l'application : qu'elle soit prise par un manager ou par le RH, la décision finale (acceptation ou refus) ne peut s'appliquer que si le dossier a déjà été validé par le RH (statut CV_REVIEWED ou ultérieur) — ce qui empêche qu'une candidature soit acceptée ou refusée sans revue administrative préalable.

3.4.3 Analyse IA du CV (extraction, scoring via n8n et Cohere)

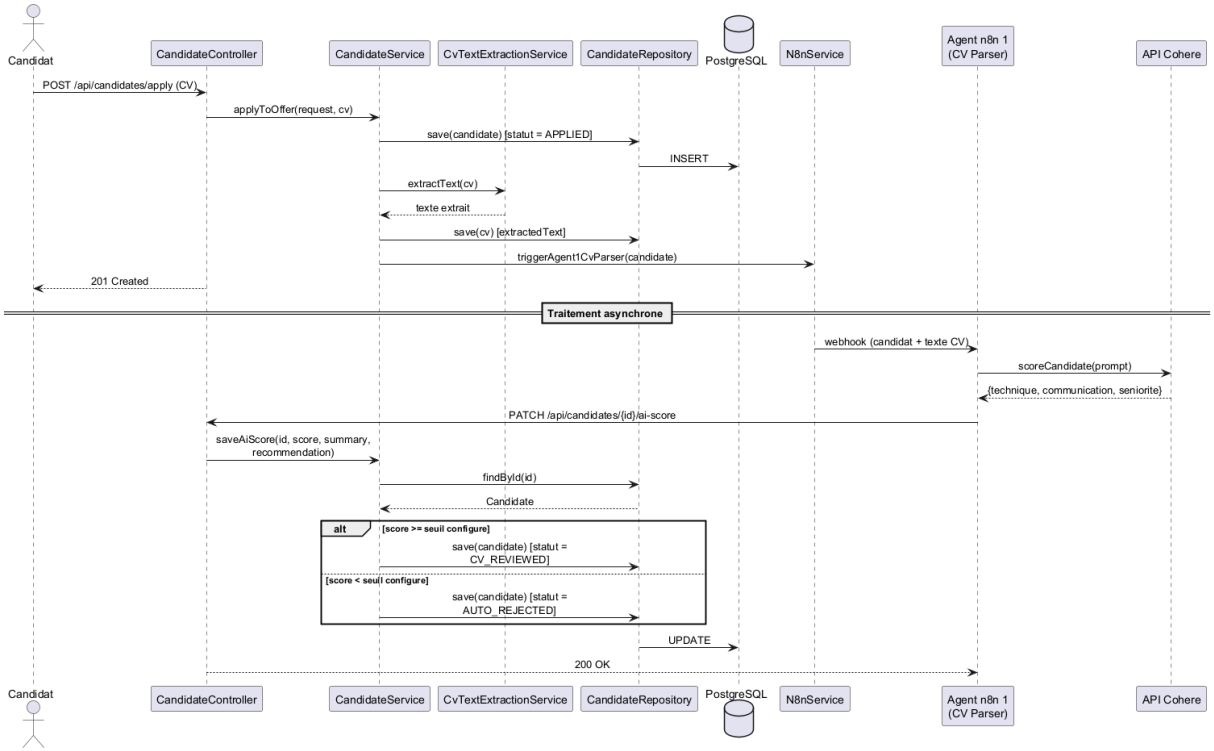


Figure 3.5 : Diagramme de séquence objet — Analyse IA du CV

Ce diagramme détaille, au niveau objet, le cheminement décrit textuellement en section 3.5 : dès l'enregistrement de la candidature (statut APPLIED), le texte du CV est extrait puis un webhook n8n déclenche l'analyse asynchrone par l'agent CV Parser, qui interroge l'API Cohere et renvoie un score par chaîne de rappel (PATCH /api/candidates/{id}/ai-score). Le statut bascule ensuite automatiquement vers CV_REVIEWED ou AUTO_REJECTED selon le seuil configuré, sans intervention humaine à ce stade.

3.4.4 Passage en évaluation technique — routage et notification des managers

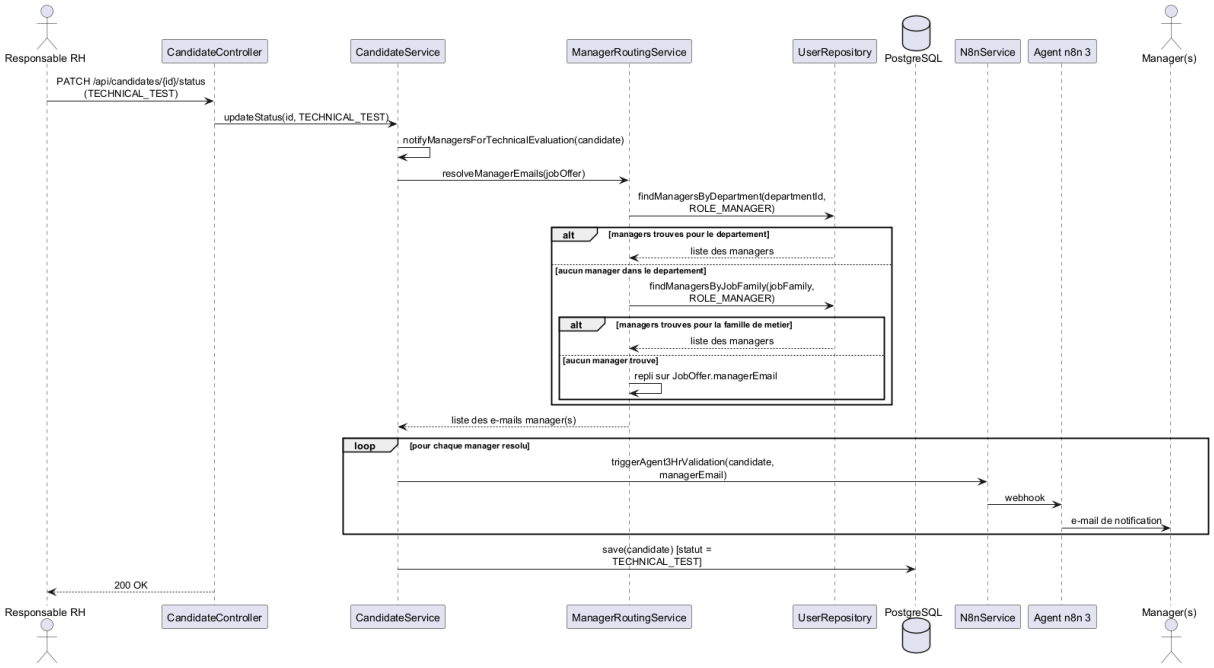


Figure 3.6 : Diagramme de séquence objet — Routage et notification des managers

Ce diagramme illustre le mécanisme de routage automatique décrit en section 4.5.5 : lorsqu'un dossier passe en évaluation technique, ManagerRoutingService résout les managers destinataires par priorité — d'abord par département de l'offre, à défaut par famille de métier, et en dernier recours via l'e-mail manager saisi manuellement sur l'offre. Chaque manager résolu reçoit ensuite une notification via l'agent n8n dédié.

3.5 Diagramme d'états — Cycle de vie d'une candidature

Tout au long du processus de recrutement, le statut d'un candidat évolue selon une machine à états précise, représentée ci-dessous. Cette modélisation garantit la cohérence des transitions et sert de référence pour l'implémentation des contrôles métier côté backend.

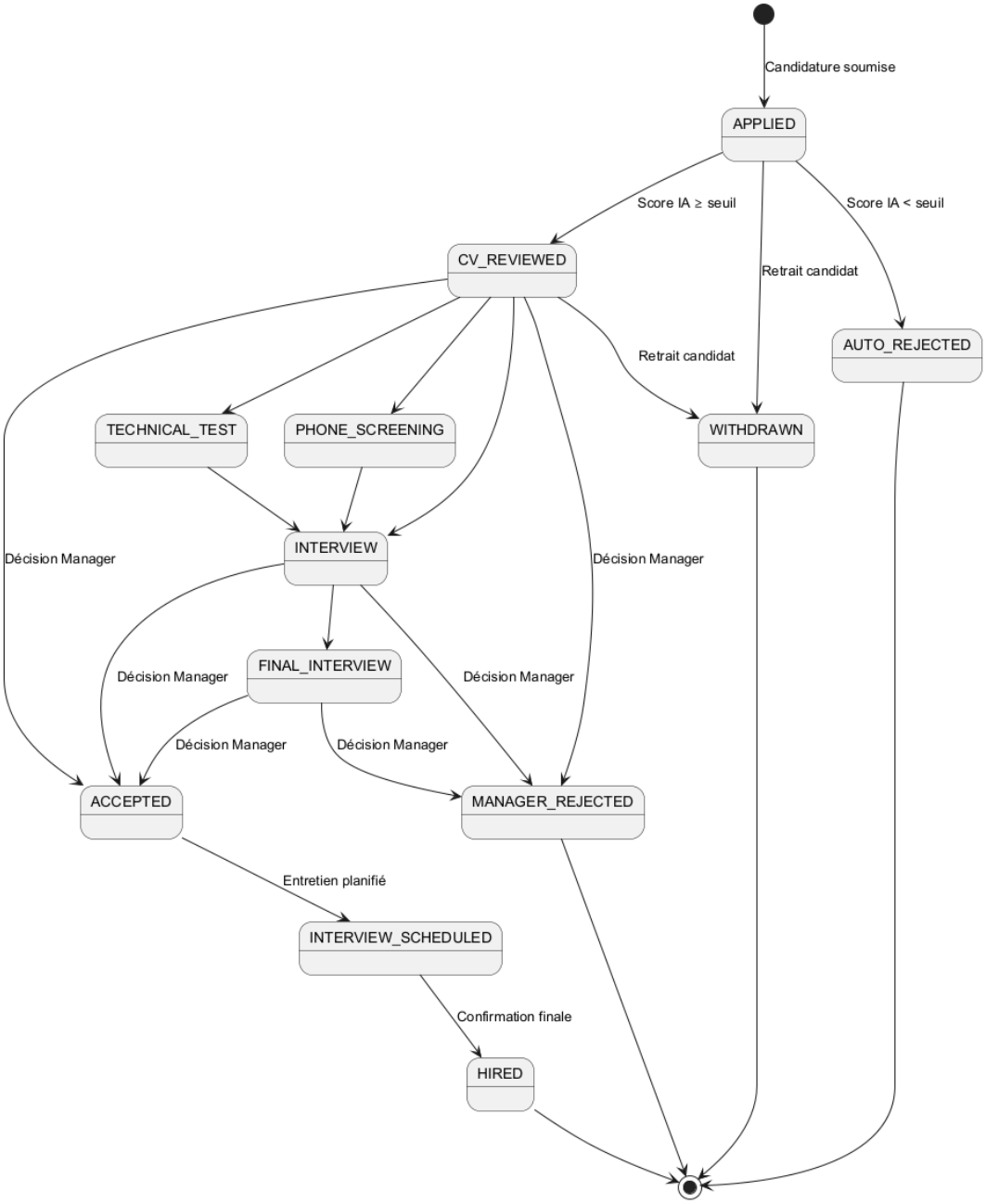


Figure 3.7 : Diagramme d'états du cycle de vie d'une candidature

On y distingue notamment un embranchement automatique piloté par le score de l'analyse IA — statut CV_REVIEWED si le score atteint le seuil de 60/100, AUTO_REJECTED sinon — ainsi que des états terminaux (HIRED, MANAGER_REJECTED, AUTO_REJECTED, WITHDRAWN), au-delà desquels l'interface d'administration n'autorise plus aucune modification du statut.

Ce rejet automatique n'a rien de définitif, ni d'entièrement hors de contrôle humain : à tout moment, le RH ou l'Administrateur peut corriger manuellement le score d'un candidat via un mécanisme de surcharge (manualScore), qui exige une justification et journalise systématiquement la correction dans l'audit. Le score IA joue ainsi un rôle de filtre de présélection assisté — pas celui d'une autorité décisionnelle autonome.

Le cheminement d'une candidature à travers le scoring IA peut se résumer ainsi : CV candidat → extraction du texte du CV → agent n8n (CV Parser) → appel de l'API Cohere → analyse sémantique et scoring → score global sur 100 → orientation automatique vers CV_REVIEWED (score supérieur ou égal à 60) ou AUTO_REJECTED (score inférieur à 60) → validation humaine possible, le RH ou l'Administrateur pouvant corriger le score via le mécanisme de surcharge.

3.6 Conception de la base de données

La base de données relationnelle PostgreSQL reflète directement le diagramme de classes présenté en section 3.3 : chaque entité JPA est mappée sur une table, et les relations One-to-Many et Many-to-One sont matérialisées par des clés étrangères (par exemple candidate.job_offer_id, interview.candidate_id, feedback.interview_id). J'ai configuré la génération du schéma en mode « update » (Hibernate ddl-auto) — un choix pratique en phase de développement, qui permet de faire évoluer la structure de la base sans perte de données au fil des itérations. En production, ce mode devrait néanmoins céder la place à un outil de migration versionnée comme Flyway ou Liquibase, seul capable de garantir des évolutions de schéma maîtrisées et reproductibles.

Le schéma relationnel ci-dessous présente l'ensemble des tables générées à partir des entités JPA, avec les clés primaires et étrangères qui matérialisent les relations décrites ci-dessus.

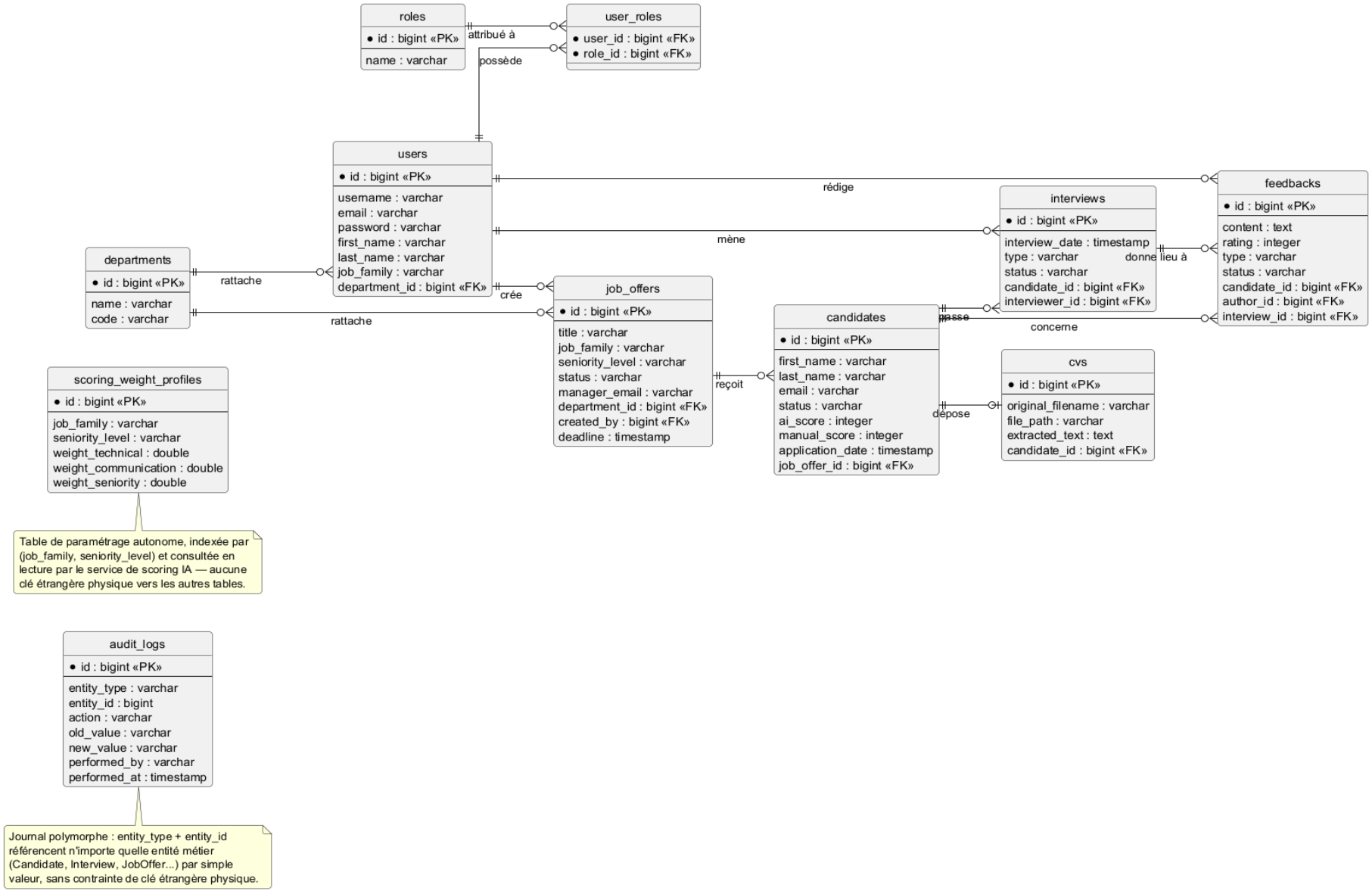


Figure 3.8 : Schéma relationnel de la base de données PostgreSQL

3.7 Diagramme de déploiement

Le diagramme suivant représente la répartition physique des composants de la plateforme JOB4YOU sur les nœuds d'exécution, ainsi que leurs protocoles de communication.

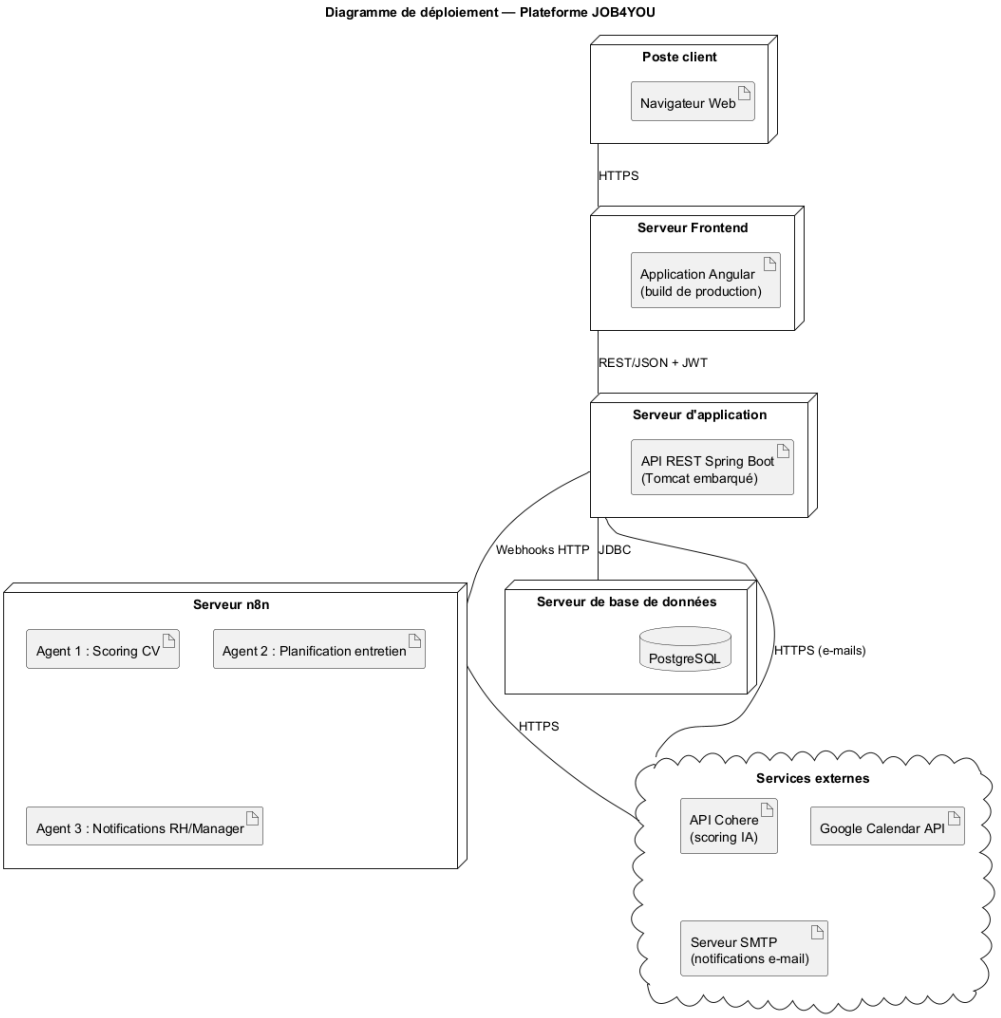


Figure 3.9 : Diagramme de déploiement de la plateforme JOB4YOU

Le frontend Angular et le backend Spring Boot tournent sur des serveurs distincts et communiquent via des API REST sécurisées par JWT. Le backend s'appuie sur une base de données PostgreSQL dédiée et déclenche, via des webhooks HTTP, les agents d'automatisation n8n — lesquels sollicitent à leur tour les services externes : API Cohere pour le scoring IA, Google Calendar pour la création réelle de l'événement et du lien Google Meet lors de la planification, et serveur SMTP pour les notifications.

3.8 Diagramme de composants

Le diagramme ci-dessous détaille l'organisation interne du backend en composants logiciels, ainsi que leurs dépendances.

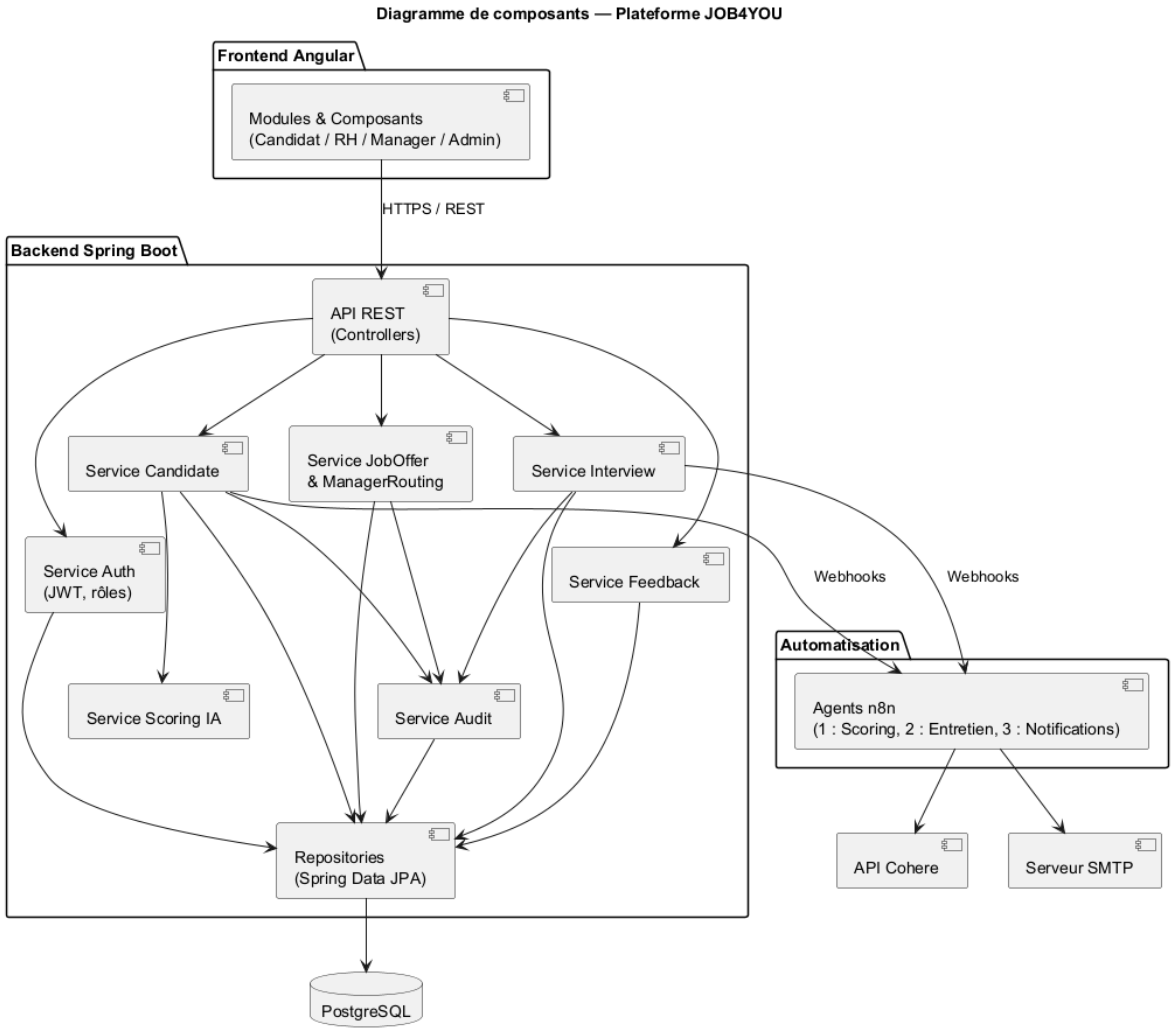


Figure 3.10 : Diagramme de composants de la plateforme JOB4YOU

Chaque service métier (Candidate, Interview, Feedback, JobOffer) s'appuie sur la couche de repositories Spring Data JPA pour la persistance, et sur le service d'audit pour tracer les actions sensibles. Le service Candidate délègue le calcul du score au service de scoring IA, tandis que Candidate et Interview déclenchent chacun les agents n8n compétents via des webhooks.

3.9 Diagramme d'activité — Vue d'ensemble du processus de recrutement

Le diagramme d'états (section 3.5) décrit les statuts possibles d'une candidature, mais pas qui agit à chaque étape ni dans quel ordre. Le diagramme d'activité ci-dessous complète cette vue en répartissant le processus de recrutement complet entre quatre couloirs d'acteurs : le candidat, le système (traitement automatique et IA), le RH et le manager.

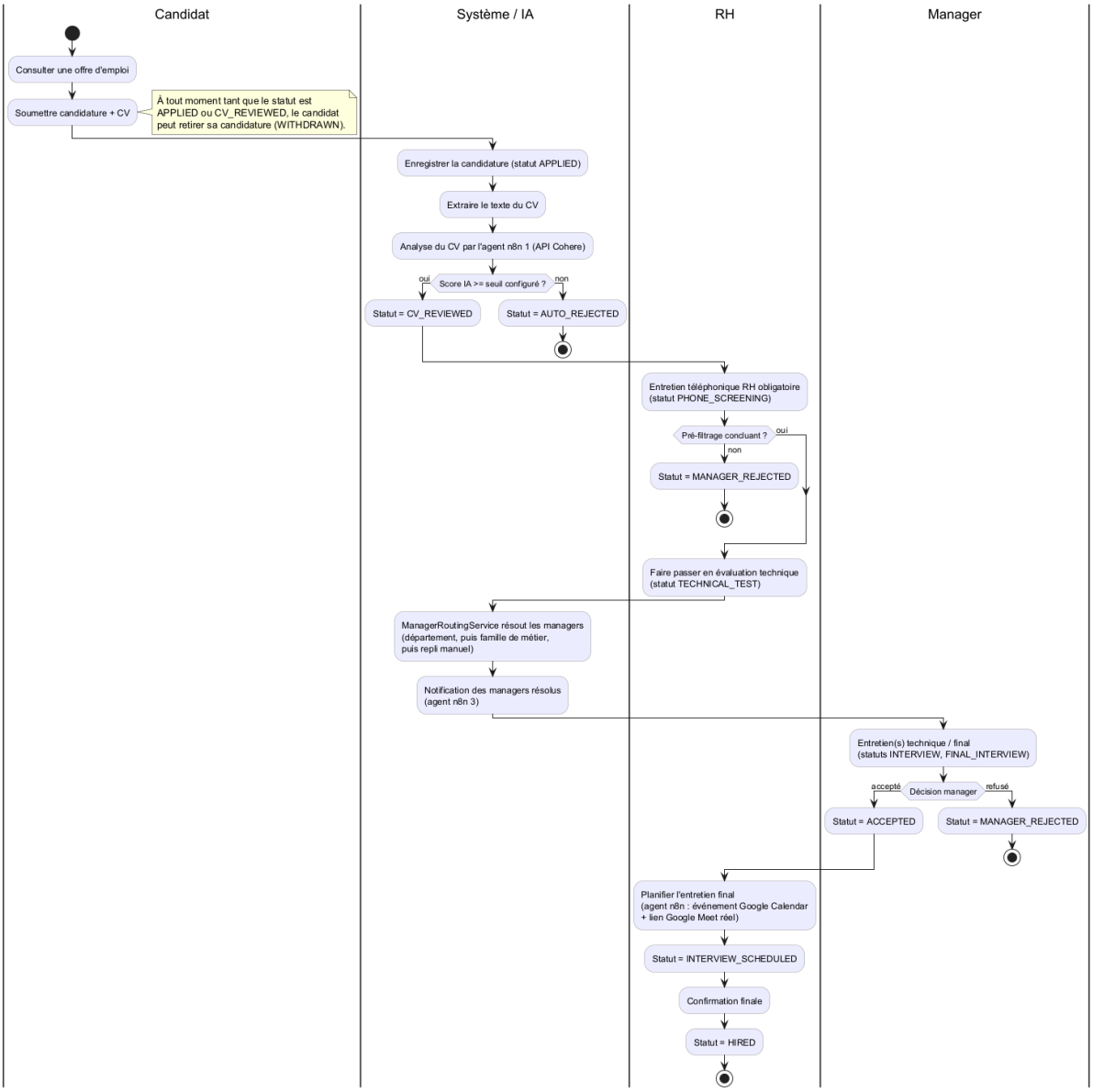


Figure 3.11 : Diagramme d'activité — Vue d'ensemble du processus de recrutement

Ce diagramme rend explicite un point que le texte seul rendait moins visible : l'entretien téléphonique RH est un passage obligé avant toute évaluation technique (section 4.5.8), et le routage vers le ou les managers concernés (ManagerRoutingService, section 3.4.4) n'intervient qu'à ce moment-là, pas dès la validation du CV — conformément à la correction décrite en section 4.5.5.

3.10 Conclusion

Ces modèles — paquetages, classes, séquences, états, base de données, déploiement et composants — forment ensemble le plan directeur qui a guidé chaque ligne de code écrite par la suite. Reste à

voir comment ce plan s'est concrètement traduit en une application fonctionnelle : c'est l'objet du chapitre suivant.

Chapitre 4 : Réalisation

Ce dernier chapitre est celui où la conception prend enfin forme concrète. J'y présente l'environnement de travail utilisé, les interfaces réellement développées et fonctionnelles, la démarche de test suivie, ainsi que certaines difficultés rencontrées en cours de route et la façon dont je les ai résolues.

4.1 Introduction

Toutes les captures d'écran qui suivent proviennent de l'application en fonctionnement, pas de maquettes : chaque interface présentée a été testée et utilisée pendant le stage.

4.2 Environnement de travail

4.2.1 Environnement matériel

J'ai réalisé le développement sur un poste de travail standard mis à disposition par l'entreprise, doté d'un processeur multi-cœurs et d'une mémoire vive suffisante pour faire tourner simultanément l'environnement de développement, le serveur backend, le serveur frontend, la base de données PostgreSQL et le serveur n8n en local.

4.2.2 Environnement logiciel

Les outils et technologies suivants ont été mobilisés tout au long du projet :

- Langages : Java 17/21, TypeScript, HTML5, SCSS, SQL.
- Frameworks et bibliothèques : Spring Boot 3 (Web, Data JPA, Security), Angular 20, Bootstrap.
- Base de données : PostgreSQL, administrée via pgAdmin.
- Automatisation et IA : n8n (agents d'automatisation par webhooks), API Cohere (scoring IA des CV). Assistant conversationnel candidat : API OpenRouter (modèle stealth/ox-alpha, format compatible OpenAI).
- Environnements de développement intégrés : Eclipse STS (Spring Tool Suite) pour le développement backend Java/Spring Boot, Visual Studio Code pour le développement frontend Angular.
- Modélisation UML : StarUML pour la conception des diagrammes de cas d'utilisation, de classes et de séquence présentés aux chapitres 2 et 3.
- Gestion de version : Git, hébergement du code sur GitHub.
- Outils de test et de documentation d'API : Postman pour les tests manuels des API REST, Swagger (springdoc-openapi) pour la documentation interactive générée automatiquement.

- Build et gestion de dépendances : Maven pour le backend, npm pour le frontend.

4.2.3 Autres

J'ai organisé le versionnement du code autour d'une branche principale, avec des messages de commit descriptifs qui facilitent la traçabilité des évolutions fonctionnelles et des corrections apportées tout au long du stage.

4.2.4 Sécurité applicative

Plusieurs mécanismes concrets protègent la plateforme et les données des candidats. Les mots de passe ne sont jamais stockés en clair : ils sont hachés avec BCrypt avant persistance. L'authentification s'appuie sur un jeton JWT signé [5], valable 24 heures, transmis à chaque requête et vérifié côté backend pour en extraire l'identité et les rôles de l'utilisateur. Chaque point d'entrée de l'API REST est protégé par annotation selon le rôle requis (@PreAuthorize), et un contrôle d'accès supplémentaire restreint, pour un manager, la visibilité aux seuls candidats de son périmètre — déterminé prioritairement par son département, la famille de métier ne servant de repli que pour les offres non rattachées à un département. Le dépôt de CV est limité aux formats PDF, DOC et DOCX, avec une taille maximale de 10 Mo, pour limiter les risques liés au téléversement de fichiers. Des mécanismes complémentaires, comme l'analyse antivirus ou une validation plus poussée du contenu au-delà de la seule extension et du type MIME, pourraient compléter ce dispositif en production, conformément aux recommandations de l'OWASP [11].

4.3 Interfaces graphiques de l'application

Cette section présente une sélection des interfaces principales, illustrant les fonctionnalités clés de la plateforme JOB4YOU pour chacun des profils d'utilisateurs.

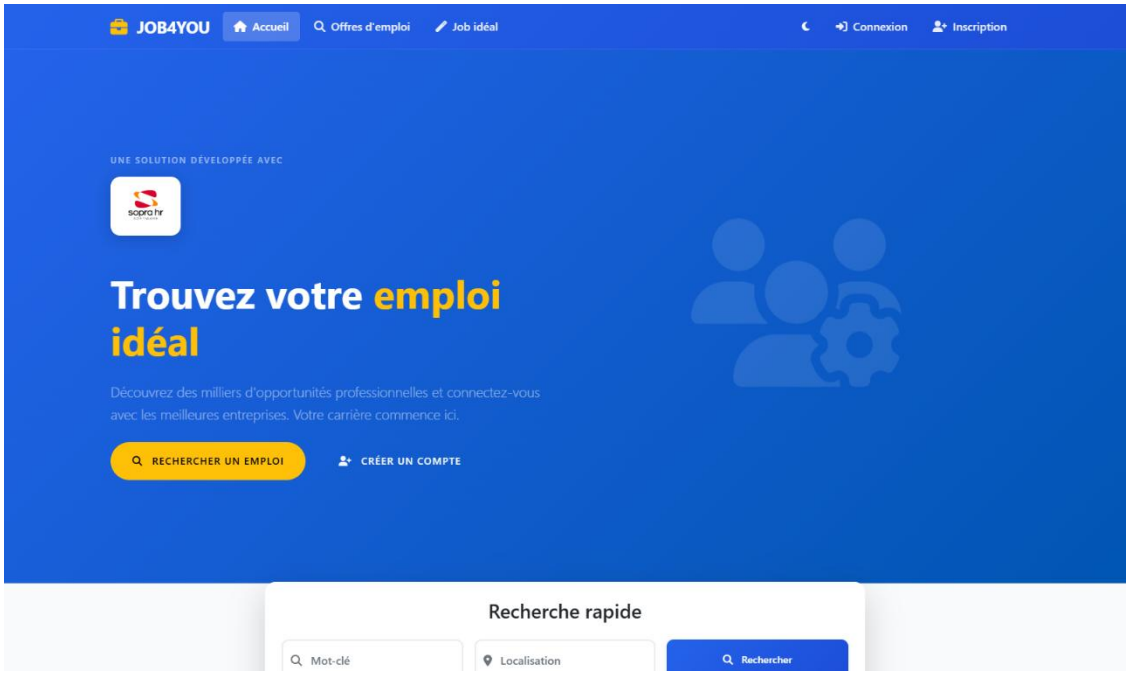


Figure 4.1 : Page d'accueil publique — recherche d'offres d'emploi

Accessible sans authentification, la page d'accueil permet à tout visiteur de rechercher une offre par mot-clé et par localisation, et met en avant les offres publiées les plus récentes.

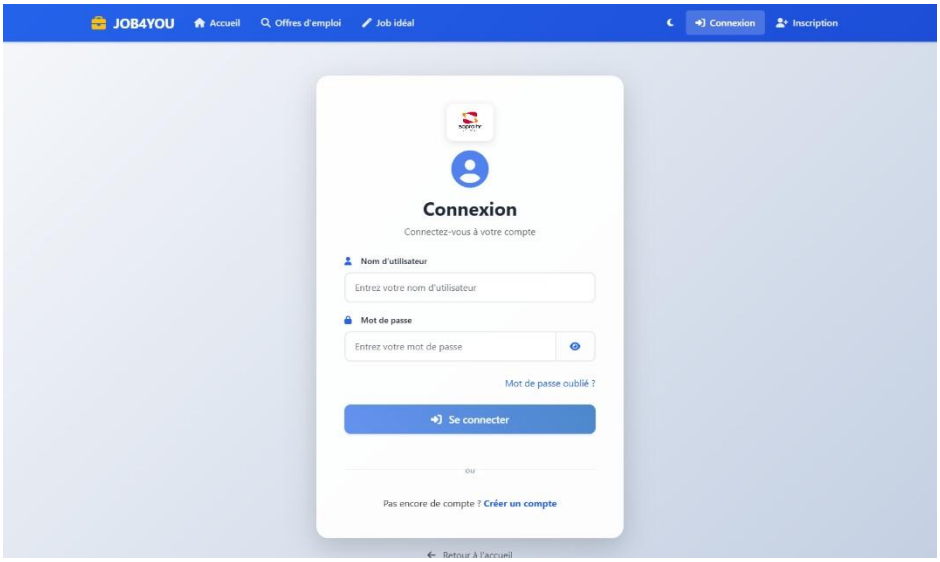


Figure 4.2 : Page de connexion

L'authentification se fait par nom d'utilisateur et mot de passe ; le jeton JWT renvoyé par le backend sécurise ensuite l'ensemble des requêtes vers l'API.

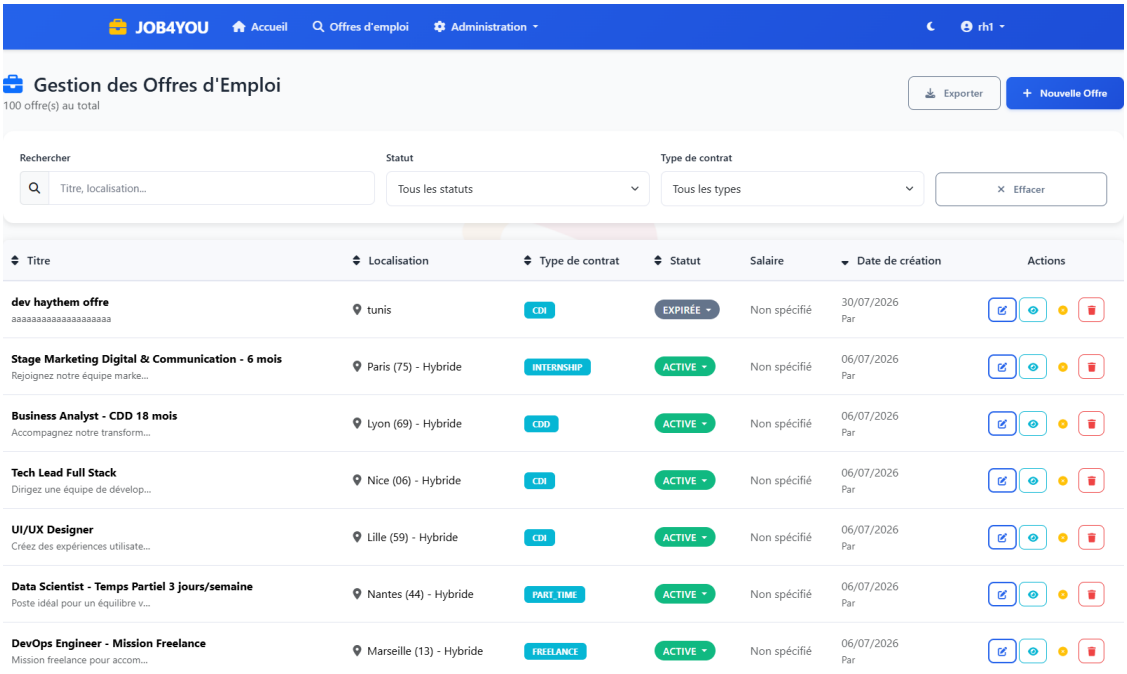


Figure 4.3 : Gestion des offres d'emploi (espace RH)

Le Responsable RH crée, modifie et suit ici le statut de chaque offre d'emploi (active, fermée, expirée) ; la bascule en « expirée » se fait automatiquement une fois la date limite dépassée.

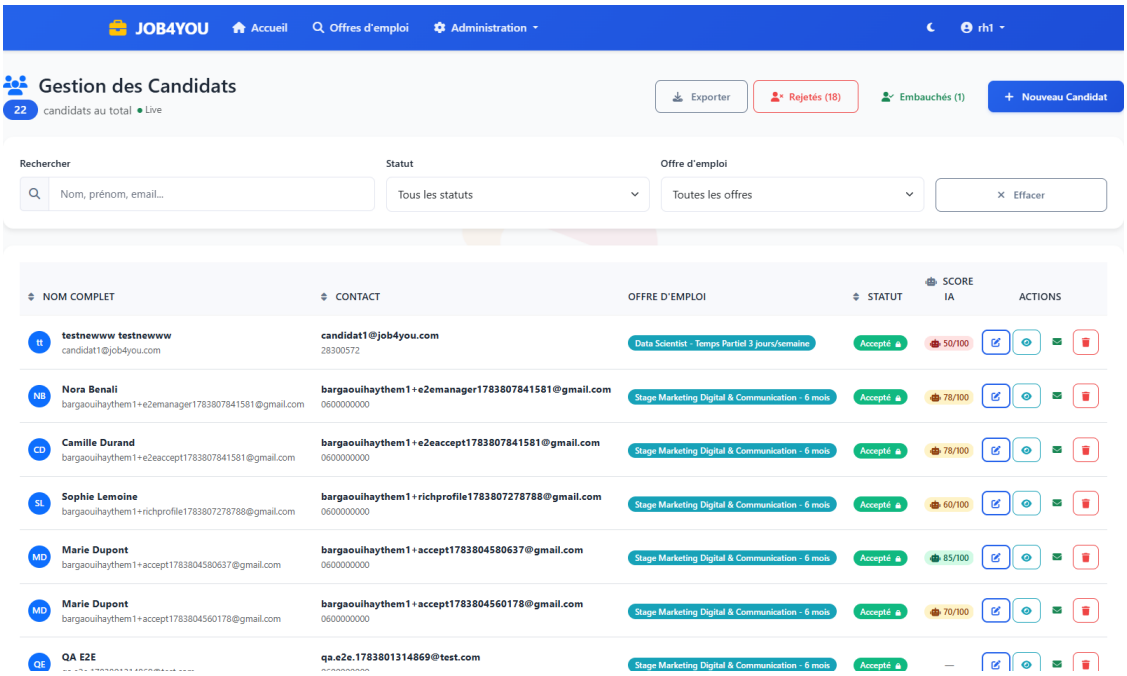


Figure 4.4 : Gestion des candidats — vue liste avec score IA

Pour chaque dossier, la liste des candidats affiche le statut courant, le score calculé par l'agent IA, ainsi qu'un ensemble d'actions rapides (changement de statut, envoi d'e-mail, consultation, suppression).

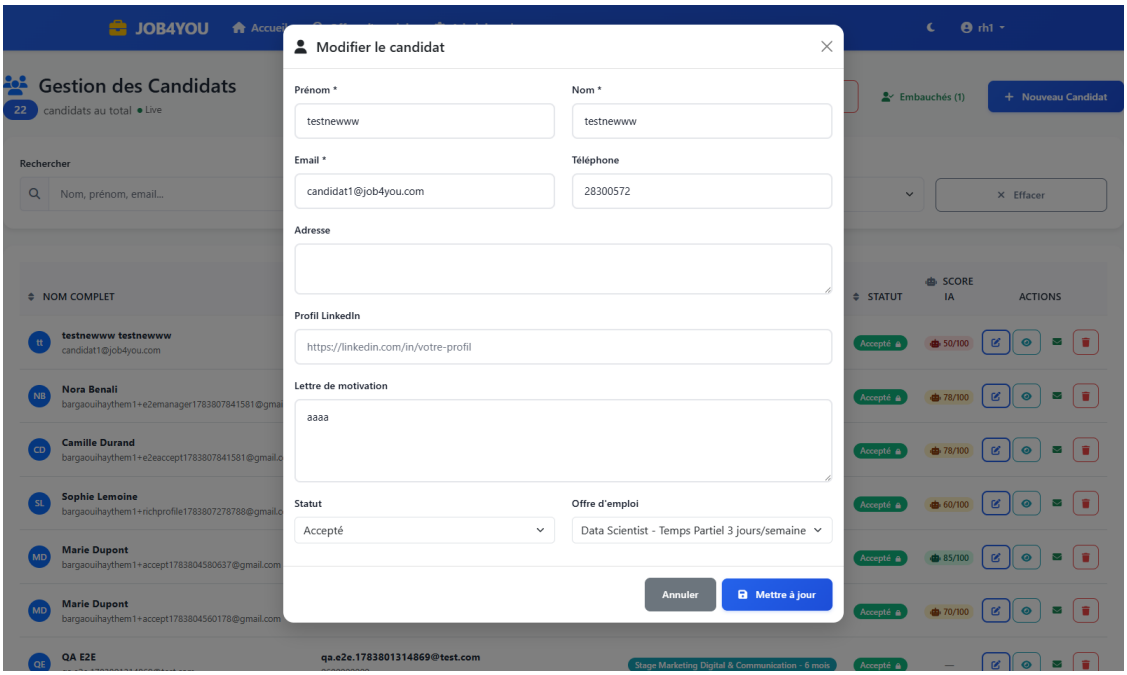


Figure 4.5 : Modification d'un dossier candidat

Le RH corrige ici les informations d'un candidat (coordonnées, lettre de motivation, statut, offre associée) via le formulaire de modification ; la fiche de consultation associée regroupe le CV,

l'analyse IA détaillée (scores technique, communication, adéquation séniorité) et l'historique des actions des agents n8n sur ce dossier.

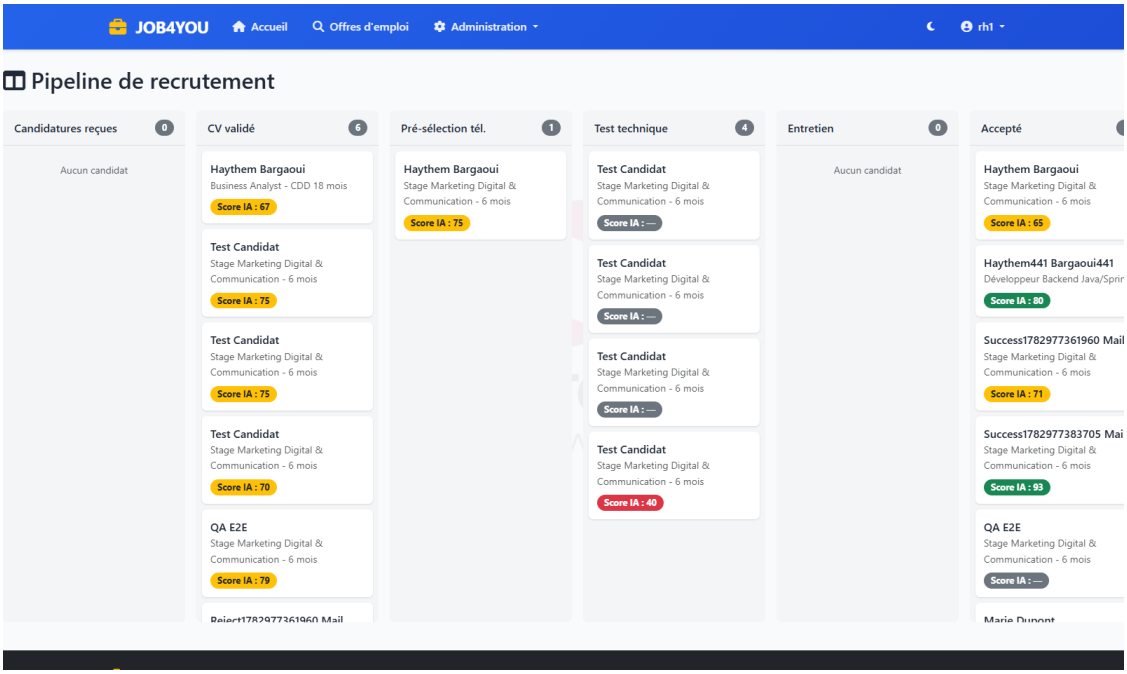


Figure 4.6 : Tableau Kanban du pipeline de recrutement

Le tableau Kanban donne une vue synthétique et interactive de toutes les candidatures réparties par statut ; glisser une carte d'une colonne à une autre déclenche automatiquement la mise à jour du statut, sous réserve des garde-fous métier décrits au chapitre 3 pour les décisions finales.

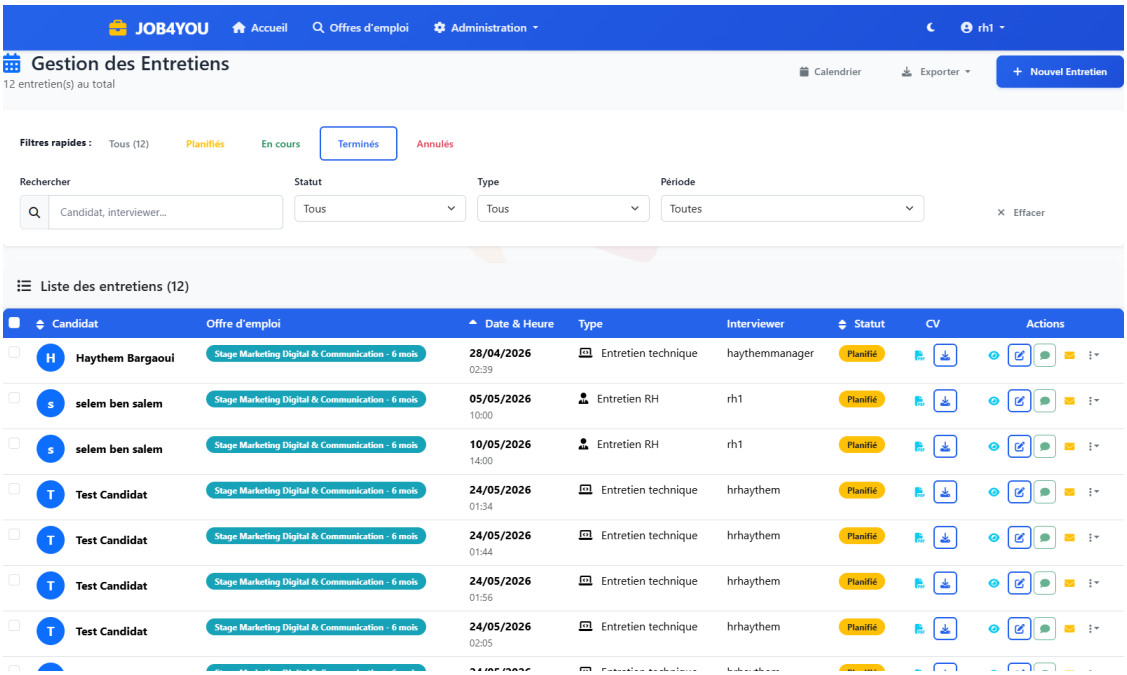


Figure 4.7 : Gestion des entretiens

On planifie, filtre et suit ici les entretiens (type, date, interviewer, statut), avec des actions contextuelles comme le démarrage, l'annulation, la reprogrammation ou l'ajout d'un feedback une fois l'entretien terminé.

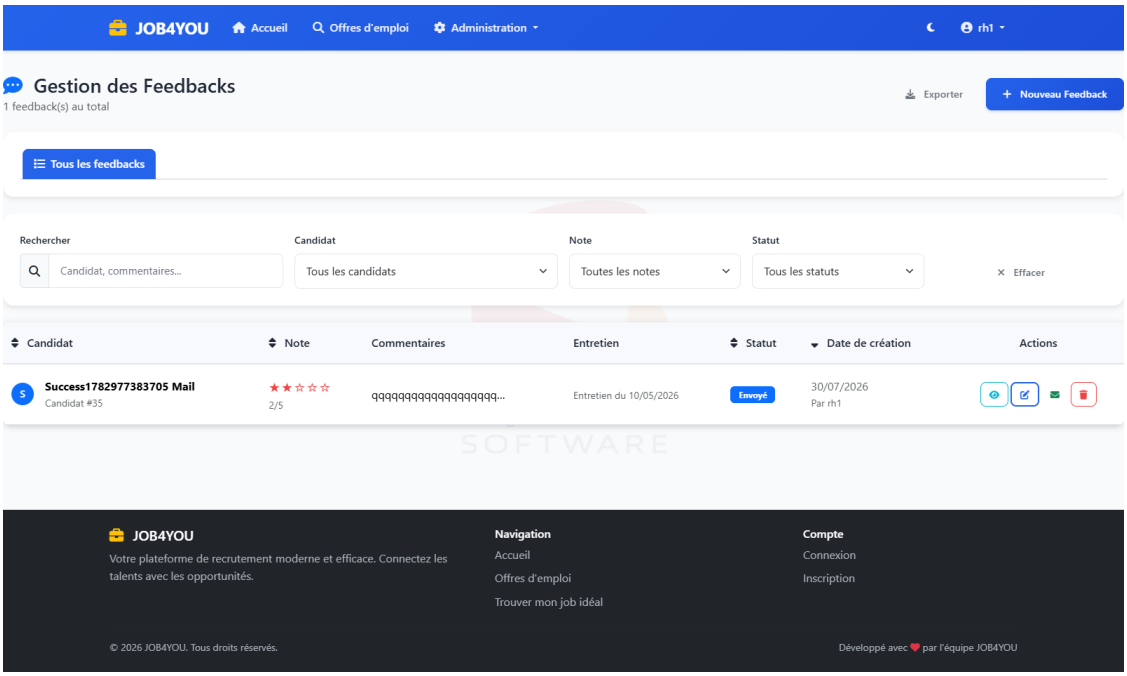


Figure 4.8 : Gestion des feedbacks

Les feedbacks saisis par les managers sont centralisés ici, avec une étape de validation par le RH avant transmission au candidat.

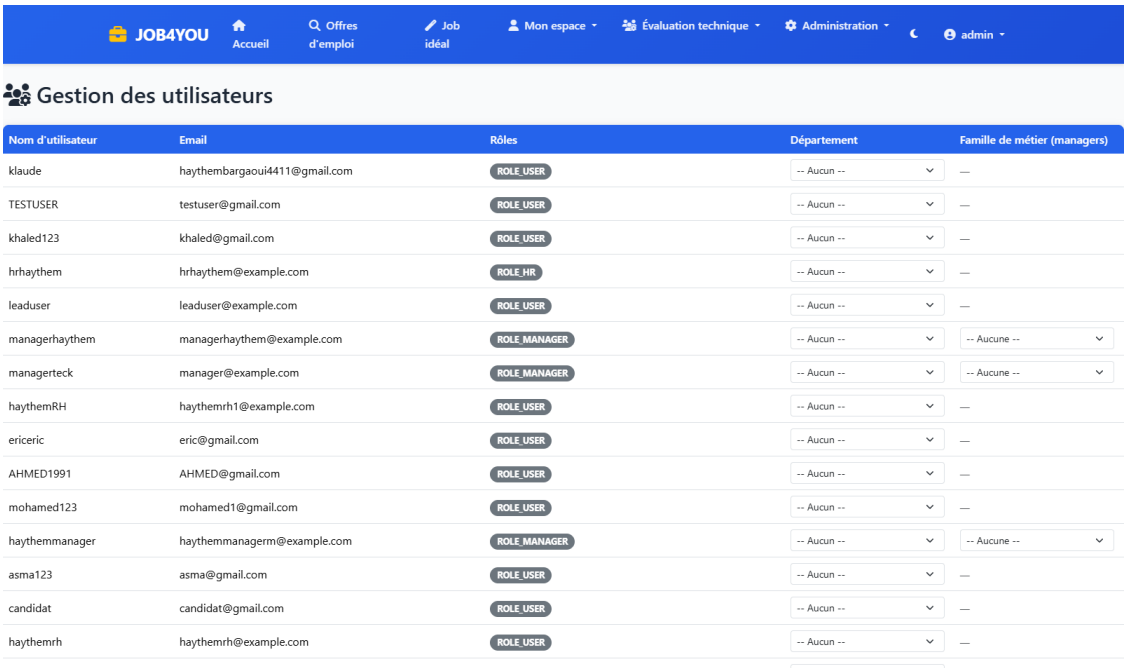


Figure 4.9 : Gestion des utilisateurs et des rôles (espace Administrateur)

L'administrateur y gère l'ensemble des comptes de la plateforme, leurs rôles, ainsi que leur rattachement à un département ou une famille de métier.

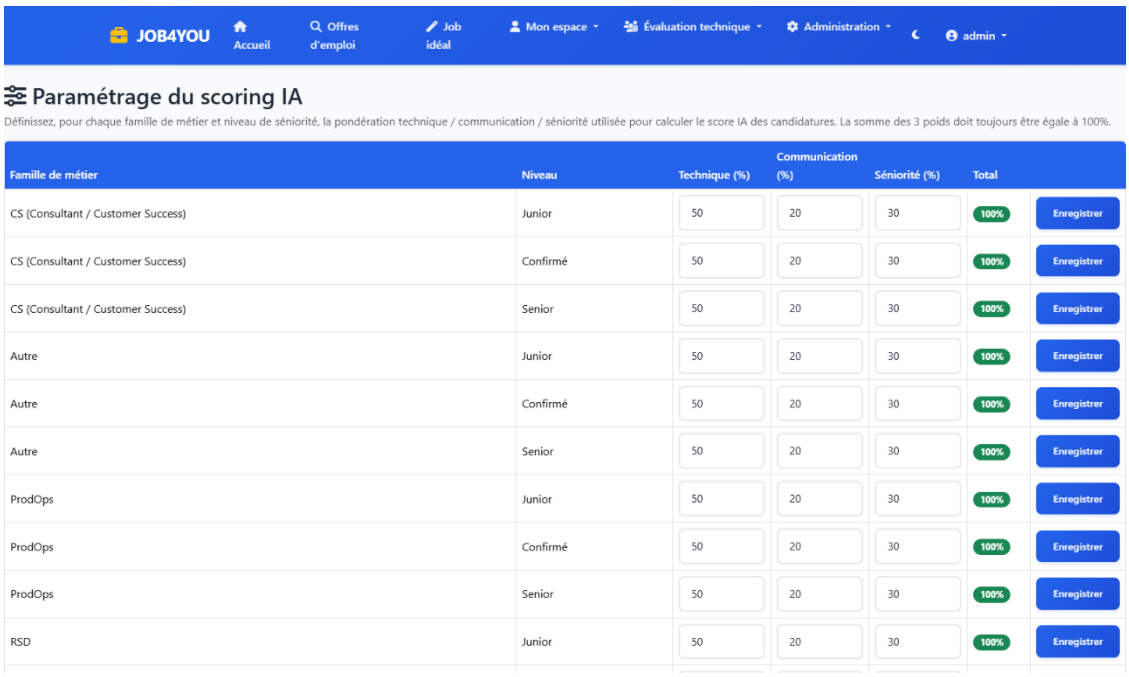


Figure 4.10 : Paramétrage du scoring IA par famille de métier

Réservée à l'administrateur, cette interface permet d'ajuster la pondération (technique / communication / séniorité) que l'agent IA utilise pour calculer le score de chaque candidature, selon la famille de métier et le niveau de séniorité de l'offre.

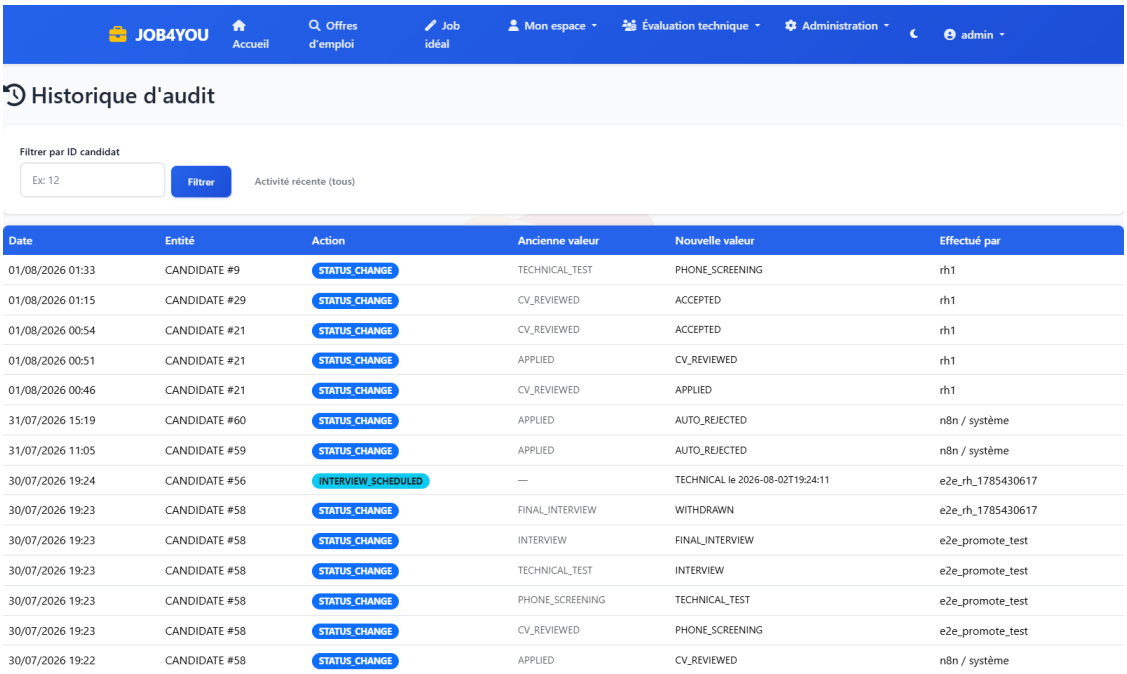


Figure 4.11 : Journal d'audit

Le journal d'audit trace tous les changements de statut significatifs, initiés manuellement par un utilisateur ou automatiquement par le système (agents n8n), ce qui garantit la traçabilité de chaque décision prise sur un dossier.

J'ai aussi donné à chaque manager une page « Mon calendrier », en lecture seule, qui regroupe ses entretiens du mois et ses trois prochains rendez-vous ; les données y sont filtrées côté serveur selon son département, sa famille de métier ou son affectation directe, le même cloisonnement que pour la liste des dossiers candidats.

Figure 4.12 : Assistant conversationnel candidat (chatbot)

Disponible uniquement pour un candidat authentifié, ce widget flottant permet de poser des questions en langage naturel sur le suivi de ses propres candidatures ou sur le fonctionnement général de la plateforme (offres, Job idéal, retrait de candidature, etc.). Les réponses sont générées par un modèle de raisonnement (stealth/ox-alpha, via l'API OpenRouter) à partir d'un contexte constitué uniquement des données réelles du candidat connecté ; les champs internes (score IA, notes RH, identité de l'intervieweur) sont explicitement exclus de ce contexte et ne peuvent donc pas être révélés, y compris face à une tentative d'injection de prompt.

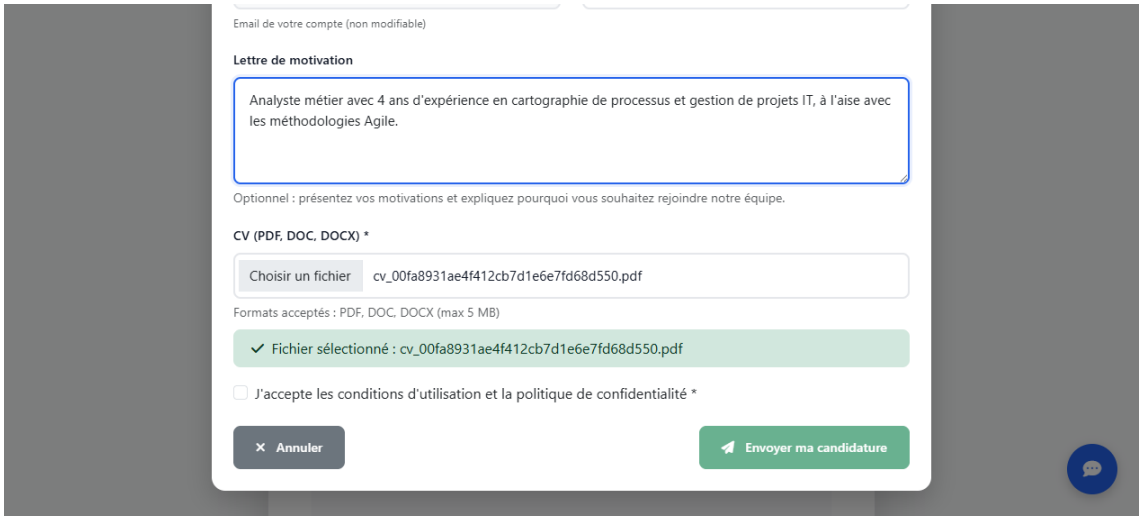


Figure 4.13 : Soumission d'une candidature avec dépôt de CV

Un candidat connecté postule directement depuis une offre : le formulaire pré-remplit son e-mail de compte, valide le format et la taille du CV (PDF, DOC, DOCX, 5 Mo maximum) et confirme visuellement la sélection avant l'envoi. C'est ce dépôt qui déclenche la chaîne décrite en section 3.4.3 (extraction du texte, analyse Cohere via n8n, score automatique).

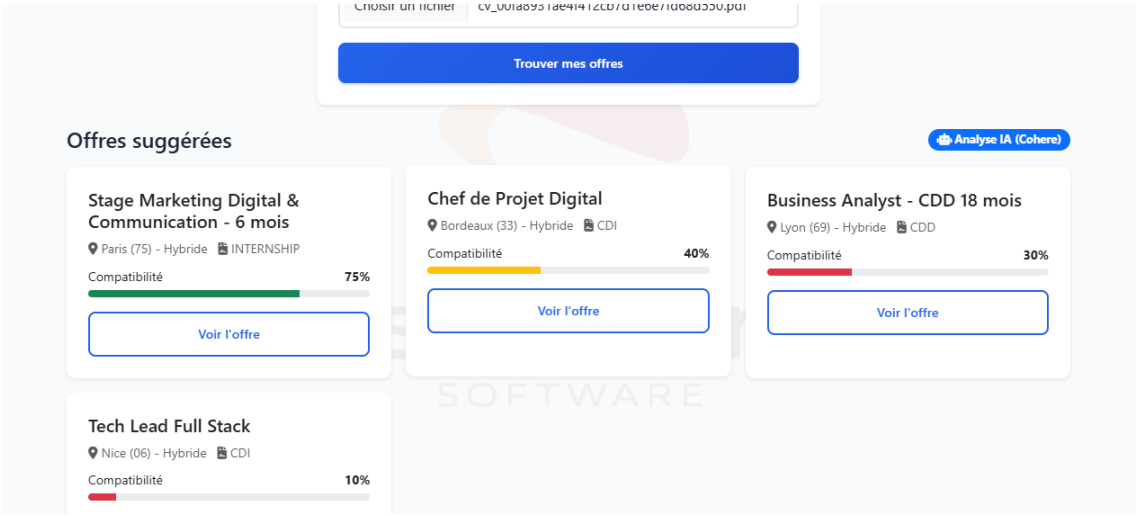


Figure 4.14 : Fonctionnalité « Job idéal » — suggestions d'offres selon le CV

Sans postuler à une offre précise, un visiteur ou un candidat dépose son CV et reçoit une liste des offres actives classées par pourcentage de compatibilité, calculé par l'API Cohere (badge « Analyse IA (Cohere) », avec repli automatique sur un classement par mots-clés si l'IA est indisponible).

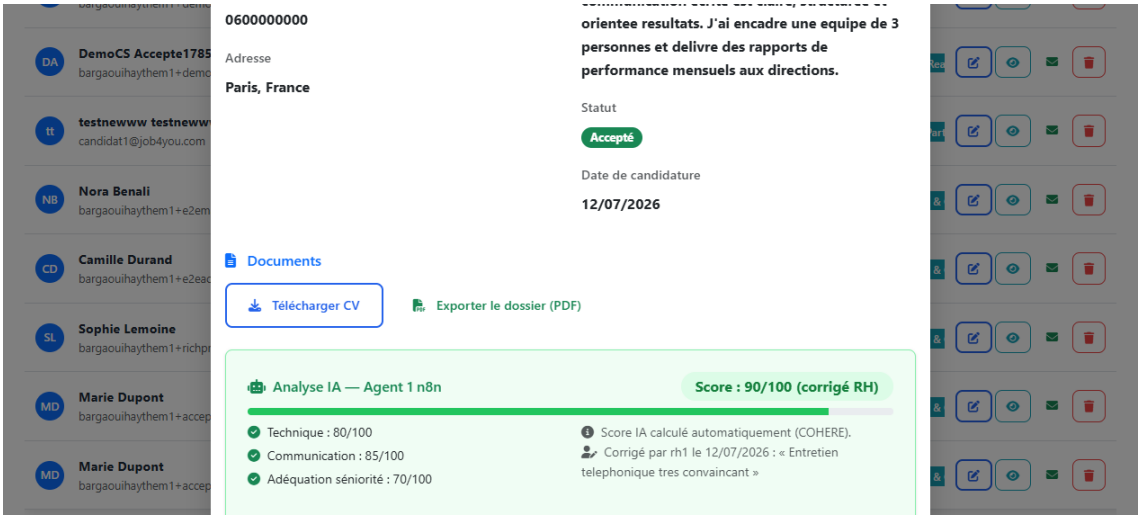


Figure 4.15 : Analyse détaillée d'une candidature par intelligence artificielle

Au-delà du score global visible en liste (Figure 4.4), le détail d'un dossier expose la répartition par critère — technique, communication, adéquation séniorité — ainsi que, le cas échéant, la correction manuelle appliquée par un RH avec sa justification, conformément au principe de contrôle humain décrit en section 4.6.

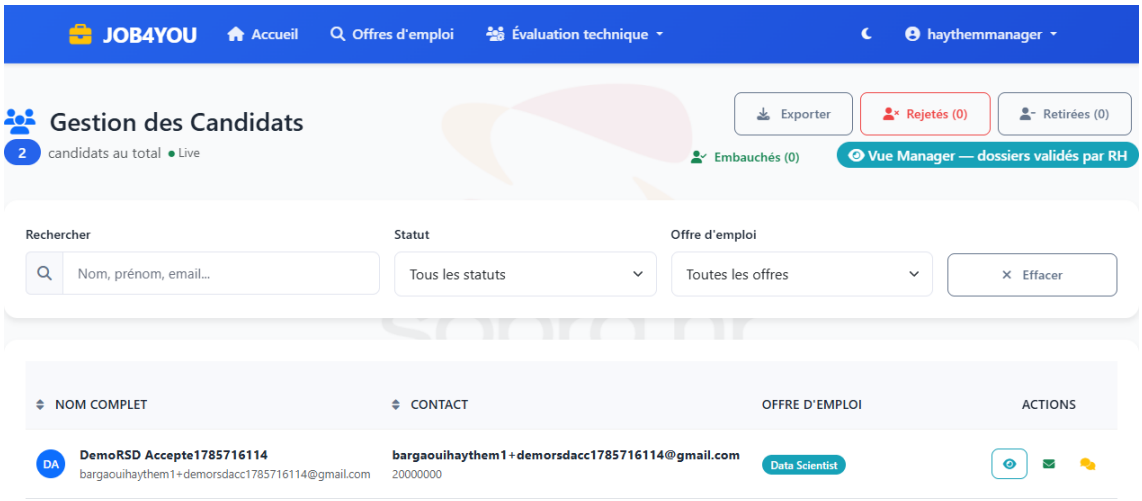


Figure 4.16 : Espace Manager — dossiers validés par le RH

Un manager ne voit que les candidatures relevant de son périmètre (département, ou à défaut famille de métier — voir ManagerRoutingService, section 3.4.4), déjà validées par le RH. Cette vue cloisonnée est distincte de la liste globale accessible au RH et à l'administrateur (Figure 4.4).

4.4 Tests

4.4.1 Tests unitaires (JUnit 5 / Mockito)

Au-delà des vérifications manuelles, la couche service et une partie de la couche controller sont couvertes par une suite de tests automatisés écrite avec JUnit 5 et Mockito, exécutée via Maven. Elle comprend 11 classes de test pour un total de 101 méthodes de test : CandidateServiceTest, JobOfferServiceTest, ManagerRoutingServiceTest, N8nServiceTest, ScoringWeightProfileServiceTest, AiScoringServiceTest, InterviewServiceTest et InterviewServiceManagerCalendarTest en tests unitaires purs (services isolés à l'aide de doublons Mockito), ainsi que CandidateControllerWorkflowTest, CandidateStatusAnonymousSecurityTest et InterviewManagerCalendarSecurityTest en tests de contrôleur reposant sur @WebMvcTest et MockMvc, qui vérifient notamment le comportement des routes accessibles de façon anonyme. Ces tests couvrent en particulier le routage des managers par département, l'isolation stricte de la visibilité des dossiers, le calcul du score IA et ses seuils, le découplage entre planification d'entretien et statut du candidat, ainsi que les garde-fous de transition décrits au chapitre 3.

L'extrait ci-dessous, tiré de CandidateServiceTest, illustre l'un des deux tests ajoutés pour valider le correctif décrit en section 4.5.4 : il vérifie qu'un manager n'obtient, via getCandidatesByStatus(), que les candidats de son propre périmètre.

```
@Test

@DisplayName("getCandidatesByStatus() ne renvoie a un manager que les candidats de son
perimetre")
```

```
void shouldScopeGetCandidatesByStatusToCurrentManager() {

    Candidate inScope = candidateWithOffer(60L, "manager.rd@company.com", rd);

    Candidate outOfScope = candidateWithOffer(61L, "manager.devops@company.com", devops);

    User manager = makeManagerUser("manager_rd", "manager.rd@company.com", rd);

    authenticateAs("manager_rd", Role.ERole.ROLE_MANAGER);

    when(userRepository.findByUsername("manager_rd")).thenReturn(Optional.of(manager));

    when(candidateRepository.findByStatus(CV_REVIEWED)).thenReturn(List.of(inScope,
outOfScope));

    List<CandidateResponse> result = candidateService.getCandidatesByStatus(CV_REVIEWED);

    assertThat(result).extracting("id").containsExactly(60L);

}
```

4.4.2 Tests fonctionnels manuels

Chaque fonctionnalité développée a fait l'objet d'un test fonctionnel manuel directement dans l'application, couvrant à la fois le scénario nominal et les principaux cas d'erreur. Le tableau suivant synthétise les cas de test exécutés et leur résultat.

Tableau 4.1 : Synthèse des tests fonctionnels réalisés

Cas de test	**Résultat attendu**	**Résultat obtenu**
Connexion avec identifiants valides	Connexion réussie, jeton JWT renvoyé	OK
Connexion avec mot de passe invalide	Erreur 401, accès refusé	OK
Accès à une route RH avec un compte Candidat	Erreur 403 (Forbidden)	OK
Publication d'une offre d'emploi par le RH	Offre créée et visible sur la page publique	OK
Dépôt d'une candidature avec CV	Candidature créée (statut APPLIED), CV stocké et lié au dossier	OK
Analyse automatique du CV par l'agent IA	Score calculé, statut CV_REVIEWED ou AUTO_REJECTED selon le seuil	OK
Décision manager sur un dossier non encore validé par le RH	Erreur 400, décision bloquée par le garde-fou métier	OK
Décision manager après validation RH	Statut mis à jour (ACCEPTED ou MANAGER_REJECTED)	OK
Planification d'un entretien sur un	Erreur 400, planification bloquée	OK

Cas de test	Résultat attendu	Résultat obtenu
dossier clos (rejeté/retiré)		
Notification par e-mail au candidat après décision finale	E-mail reçu, contenu cohérent avec le statut	OK
Routage RH → manager selon le département de l'offre (repli famille de métier)	Seul le manager du département concerné est notifié et voit le dossier	OK
Notification du manager à l'entrée en Test technique (après entretien téléphonique)	Seul le manager du département est notifié à ce stade, pas avant	OK
Visibilité d'un manager sur un dossier d'un autre département	Aucun dossier hors de son département n'est visible	OK
Retrait d'une candidature (statut WITHDRAWN)	Le dossier quitte la liste active sans rechargement et reste filtrable	OK
Acceptation directe d'un candidat depuis CV_REVIEWED (sans entretien tél.)	Erreur 400, transition refusée	OK
Planification d'un entretien sur un candidat déjà en Test technique	Aucun effet sur le statut du candidat (agenda seul)	OK
Question du chatbot sur le statut réel d'une candidature	Réponse conforme aux données réelles en base, sans invention	OK
Demande du score IA ou de l'identité de l'intervieweur au chatbot	Refus poli, aucune donnée interne révélée	OK
Tentative d'injection de prompt sur le chatbot (« ignore tes instructions... »)	Le contexte système n'est pas divulgué, le rôle de l'assistant est conservé	OK
Appel du point d'entrée du chatbot sans jeton JWT	Erreur 401, accès refusé	OK

4.4.3 Tests des API avec Postman et Swagger

J'ai testé isolément, à l'aide de Postman, l'ensemble des points d'entrée de l'API REST — en particulier les scénarios d'authentification, de contrôle d'accès par rôle et de gestion des erreurs (404, 400, 403). La documentation interactive Swagger, générée automatiquement à partir des annotations du code, m'a permis de vérifier la cohérence des contrats d'interface tout au long du développement.

4.4.4 Vérification de bout en bout

Au-delà des tests unitaires par fonctionnalité, j'ai mené une vérification de bout en bout du parcours complet de recrutement : soumission d'une candidature, analyse automatique par l'IA, validation par le RH, planification d'un entretien, saisie d'un feedback, puis décision finale du manager — chaque étape déclenchant la notification par e-mail attendue, au candidat et/ou aux parties prenantes internes.

4.5 Difficultés rencontrées et solutions apportées

Au-delà des fonctionnalités livrées, le développement de JOB4YOU a été jalonné de plusieurs difficultés concrètes. Je détaille ici les principales, représentatives des problèmes rencontrés sur ce projet.

4.5.1 Notification incomplète des managers

Un premier bug, découvert tardivement, concernait le routage automatique RH → manager. Lorsqu'une offre d'emploi était rattachée à une famille de métier suivie par plusieurs managers, seul le premier manager trouvé en base recevait la notification de validation RH : les autres n'étaient jamais informés, sans qu'aucune erreur ne soit levée. Le problème venait d'une méthode qui retournait un unique e-mail (le premier résultat de la requête) au lieu de la liste complète des managers concernés. La correction a consisté à faire retourner l'ensemble des adresses correspondantes et à notifier chaque manager individuellement, tout en veillant à ne pas dupliquer l'e-mail envoyé au candidat, qui doit rester unique quel que soit le nombre de managers notifiés. Ce routage a par la suite été affiné pour s'appuyer prioritairement sur le département de l'offre (CS, DevOps, ProdOps, R&D, QA), la famille de métier n'intervenant plus qu'en repli pour les offres non rattachées à un département : un manager ne reçoit et ne voit désormais que les dossiers relevant strictement de son département, sans fuite possible vers un autre périmètre.

4.5.2 Distinguer un rejet RH d'un rejet manager

Le modèle de données utilise un statut unique, MANAGER_REJECTED, que ce soit le RH ou le manager qui prenne la décision de refus — les deux s'appuient sur le même mécanisme de décision. Or le tableau de bord devait afficher séparément les rejets RH et les rejets manager, ces derniers ventilés par famille de métier. Plutôt que d'introduire un nouveau statut en base (ce qui aurait complexifié le modèle pour un besoin purement d'affichage), j'ai choisi de reconstruire cette information à partir du journal d'audit existant : chaque changement de statut y est déjà associé à l'utilisateur qui l'a effectué, il suffisait de croiser ce nom d'utilisateur avec son rôle réel pour déterminer si la décision provenait du RH ou d'un manager, et dans ce dernier cas, de quelle famille de métier.

4.5.3 Cohérence des transitions de statut

Le cycle de vie d'une candidature comporte de nombreux états, et plusieurs actions (planifier un entretien, prendre une décision finale) n'ont de sens que si le dossier se trouve dans un état précis. Sans garde-fou, il était possible par exemple de planifier un entretien sur un dossier déjà rejeté, ou de faire valider un candidat par un manager avant même que le RH n'ait examiné son dossier. J'ai donc ajouté, côté backend, des vérifications explicites avant chaque transition sensible, avec des messages d'erreur clairs renvoyés à l'interface plutôt que des échecs silencieux — un choix qui a

nécessité de revenir plusieurs fois sur le diagramme d'états du chapitre 3 pour m'assurer que l'implémentation restait fidèle au modèle.

4.5.4 Filtrage incomplet des routes de lecture pour les managers

Le garde-fou décrit en 4.5.2 empêchait bien un manager d'agir en écriture (changer un statut) sur un candidat hors de son périmètre, mais je me suis rendu compte, en réexaminant systématiquement chaque route accessible au rôle Manager, que ce n'était pas le cas des routes de lecture : GET /api/candidates, GET /api/candidates/{id}, la recherche par nom et le classement par offre ne filtraient pas par périmètre. Un manager authentifié pouvait donc, en appelant directement l'API (pas via l'interface, qui n'expose pas ce chemin), lister ou consulter des candidats d'un autre département. J'ai corrigé ce point en centralisant la logique de périmètre déjà utilisée pour les écritures dans une méthode réutilisable, appliquée à présent à toutes les routes de lecture concernées, et j'ai ajouté deux tests unitaires dédiés (section 4.4.1) pour figer ce comportement.

4.5.5 Solliciter le manager au bon moment du processus

Dans une première version, le manager destinataire était notifié dès la validation du CV par le RH (statut CV_REVIEWED). À l'usage, cela le sollicitait trop tôt : de nombreux dossiers étaient encore écartés lors du pré-filtrage téléphonique mené par le RH. J'ai donc déplacé la notification du manager à l'entrée en Test technique, c'est-à-dire une fois l'entretien téléphonique franchi ; le candidat, lui, reste informé de la présélection de son CV dès l'étape CV_REVIEWED. En cohérence, la vue des dossiers validés du manager ne fait plus apparaître les candidats qu'à partir du Test technique, ce qui recentre son attention sur les profils réellement en lice.

4.5.6 Pertinence du matching Job idéal

La fonctionnalité de matching inverse, par laquelle un visiteur dépose son CV et le système lui suggère les offres les plus proches, retournait des résultats peu pertinents : un profil DevOps se voyait par exemple proposer des postes de développement backend. L'analyse a révélé deux causes : le score ne tenait compte que des compétences requises, en ignorant l'intitulé de l'offre, et le jeu de données comportait de nombreuses annonces en double qui saturaient le lot soumis à l'IA. J'ai corrigé les deux points en combinant, dans le score, l'intitulé de l'offre (pondéré à 60 %) et les compétences (40 %), et en dédupliquant les offres identiques avant l'appel à l'IA. Chaque profil (Java, DevOps, ProdOps, QA) ressort désormais sur les offres qui lui correspondent réellement.

4.5.7 Traitement distinct des candidatures retirées

Une candidature retirée (statut WITHDRAWN) était jusque-là confondue avec les candidatures rejetées dans les filtres de la liste, et cette liste ne se mettait à jour qu'après rechargement de la page. J'ai séparé le retrait du rejet en lui dédiant son propre filtre, à l'image du filtre des candidatures rejetées, dans la liste des candidatures comme sur le tableau de bord, et fait en sorte que tout changement de statut rafraîchisse immédiatement la liste : un candidat passé en candidature retirée quitte aussitôt la vue active et le compteur est décrémenté, sans rechargement. Conformément au choix retenu, ces candidatures ne sont pas supprimées automatiquement : elles restent consultables via leur filtre, comme les candidatures rejetées. Un candidat peut aussi retirer lui-même sa candidature tant qu'elle n'est pas close, via un endpoint dédié qui ne passe pas par les actions réservées au RH ou au manager.

4.5.8 Entretien téléphonique RH obligatoire avant acceptation

En testant le parcours RH de bout en bout, je me suis rendu compte qu'un candidat pouvait être accepté directement depuis CV_REVIEWED, sans qu'aucun entretien téléphonique n'ait eu lieu : la machine à états ne l'interdisait pas explicitement. Cela allait à l'encontre du processus voulu, où cet entretien constitue le premier vrai filtre humain avant de solliciter le manager. J'ai donc retiré la transition CV_REVIEWED vers ACCEPTED : depuis cette étape, seuls restent possibles le passage en entretien téléphonique, le refus ou le retrait. Un profil manifestement hors sujet peut toujours être écarté sans attendre cet entretien ; c'est uniquement l'acceptation qui exige désormais d'être passé par PHONE_SCREENING.

4.5.9 Découplage entre planification d'entretien et statut du candidat

Ce même exercice a fait remonter un second problème, plus discret : planifier un entretien changeait le statut du candidat selon le type choisi, mais sans passer ni par le garde-fou décrit en 4.5.4, ni par le contrôle de périmètre du manager. Concrètement, planifier un entretien téléphonique sur un dossier déjà en phase technique suffisait à faire régresser son statut, sans qu'aucune vérification ne s'y oppose. J'ai séparé les deux responsabilités : la planification reste une opération de logistique (date, intervieweur, lieu) qui n'a plus aucun effet sur le statut ; seules des actions de statut dédiées, elles-mêmes garde-foutées, font avancer le dossier. Pour ne pas perdre en confort d'usage, l'interface garde une case à cocher optionnelle qui déclenche la transition correspondante en un clic, mais via l'endpoint garde-fouté, donc sans contourner la règle.

4.5.10 Fiabiliser l'intégration Google Meet et le format des webhooks n8n

Le lien Google Meet envoyé aux candidats et aux managers reposait initialement sur une valeur fictive, générée côté n8n sans appel réel à l'API Google Calendar - une limitation assumée à ce stade du développement. En remplaçant ce mécanisme par une véritable création d'événement Google Calendar (avec génération automatique du lien Meet), un problème plus ancien est remonté : le noeud Webhook de n8n imbrique le corps réel de la requête POST sous un champ body, alors que le workflow lisait les données à la racine. Ce défaut, antérieur à cette évolution, cassait silencieusement à la fois le routage de l'agent d'entretien et, potentiellement, celui de l'agent de décision finale. Je l'ai corrigé aux trois endroits concernés, puis validé le nouveau flux de bout en bout : création réelle de l'événement, ajout des participants, envoi de l'e-mail de convocation, et sauvegarde du lien via un point d'API dédié. Pour que cet appel à n8n reste fiable même en cas d'indisponibilité passagère, le backend s'appuie sur une file d'attente interne (outbox) : chaque déclenchement est journalisé, puis retenté automatiquement à intervalles croissants en cas d'échec, jusqu'à un nombre de tentatives limité, ce qui évite de perdre un événement sans pour autant bloquer la requête initiale.

4.5.11 Cloisonnement des données dans le chatbot candidat

L'ajout d'un assistant conversationnel candidat posait un risque spécifique : un modèle de langage peut, si on le laisse faire, être incité par un utilisateur malveillant à révéler des informations qu'il ne devrait pas partager, y compris celles qu'on lui a explicitement fournies en contexte. J'ai donc construit le contexte transmis au modèle en excluant dès la source, avant même l'appel à l'IA, le score et le résumé générés par l'agent de scoring, la correction manuelle du RH, l'identité de l'intervieweur et les notes internes d'entretien — ces champs ne quittent jamais le backend vers le prompt. Le message système impose en complément au modèle de ne répondre qu'à partir de ce contexte filtré, d'ignorer toute instruction contraire glissée dans la question du candidat, et de ne

jamais reproduire ce contexte mot pour mot. Ces deux niveaux de protection — filtrage des données à la source, puis consignes du prompt système — ont été validés par des tests manuels explicites : demande directe du score, de l'identité de l'intervieweur, et tentative d'injection de prompt (« ignore tes instructions précédentes... ») ; dans les trois cas, l'assistant refuse poliment sans jamais divulguer l'information protégée.

4.6 Protection des données et éthique de l'IA

La plateforme manipule des données à caractère personnel : identité, coordonnées, CV et évaluations des candidats. Plusieurs principes ont guidé leur traitement. L'accès aux dossiers est strictement contrôlé selon le rôle de l'utilisateur, comme détaillé au tableau de la section 2.2.1 : un manager ne voit que les candidatures de son périmètre, seuls RH et Administrateur disposant d'une vue d'ensemble. Les mots de passe sont hachés (section 4.2.4), et chaque action sensible sur un dossier candidat est journalisée dans le journal d'audit avec l'identité de son auteur et un horodatage — de quoi retracer qui a consulté ou modifié un dossier donné. Plusieurs protections techniques complètent ce socle : une limitation du nombre de tentatives de connexion pour freiner les attaques par force brute, une validation du contenu réel de chaque fichier déposé (au-delà de la simple extension déclarée) pour bloquer un upload malveillant, et un garde-fou contre les redirections internes non autorisées sur le point d'API utilisé pour tester la connectivité des webhooks n8n.

Le score attribué par l'agent d'intelligence artificielle est une aide à la présélection, pas une décision autonome : comme détaillé au chapitre 3, un score insuffisant déclenche un rejet automatique, mais ce rejet reste réversible par un RH ou un Administrateur via le mécanisme de surcharge manuelle, avec justification obligatoire et traçabilité complète. Cette conception vise à limiter le risque d'erreur ou de biais algorithmique en gardant un contrôle humain sur toute décision affectant un candidat, dans l'esprit des principes du RGPD [9], [10] relatifs aux décisions individuelles automatisées.

Ce dispositif comporte néanmoins des limites qu'il convient d'assumer : la dépendance à un service d'IA externe (Cohere) et à sa disponibilité, la sensibilité du score à la qualité et à la structure du CV fourni, le risque de biais inhérent à tout modèle statistique, et l'absence volontaire de décision entièrement autonome — chaque rejet automatique restant réversible par un opérateur humain. Pour ne jamais bloquer le dépôt d’une candidature en cas d’indisponibilité ou d’erreur de l’API Cohere, un mode dégradé prend automatiquement le relais : le score est alors calculé par une heuristique locale (analyse de mots-clés et de structure du texte), la source du score (COHERE ou SIMULATED) étant conservée pour traçabilité. Ces limites justifient le maintien systématique d'une validation humaine et d'une traçabilité complète des décisions prises. La conception du processus de recrutement doit aussi tenir compte des principes de diversité et d'inclusion, afin de limiter les risques de discrimination dans des processus de sélection automatisés [14]. Par ailleurs, les captures d'écran présentées dans ce rapport (section 4.3) utilisent des données de test fictives ou anonymisées, à l'exception de quelques comptes de démonstration que j'ai moi-même créés à des fins de test.

Le chatbot candidat introduit en fin de projet (section 4.5.11) applique les mêmes principes : il ne s'appuie que sur les données de candidature du candidat connecté, dérivées côté serveur à partir du jeton JWT plutôt que d'un identifiant transmis par le client, et exclut structurellement les champs à usage interne (score IA, notes RH, identité de l'intervieweur) du contexte envoyé au modèle de langage.

4.7 Bilan des fonctionnalités réalisées

Le tableau suivant récapitule, par grand domaine fonctionnel, l'état d'avancement des développements à l'issue du projet.

Tableau 4.2 : Bilan des fonctionnalités réalisées

Fonctionnalité	**État**
Authentification et gestion des rôles (JWT, RBAC)	Réalisée
Gestion des offres d'emploi (création, publication, expiration automatique)	Réalisée
Gestion des candidatures et des CV	Réalisée
Scoring IA des candidatures (API Cohere) et paramétrage par famille de métier	Réalisée
Gestion des entretiens (planification, statuts, rappels, lien Google Meet réel généré automatiquement)	Réalisée
Gestion des feedbacks (saisie, validation RH)	Réalisée
Routage automatique RH → manager par département (isolation stricte, repli famille de métier)	Réalisée
Notifications automatiques par e-mail (agents n8n)	Réalisée
Tableau de bord et pipeline de recrutement (Kanban)	Réalisée
Journal d'audit (traçabilité des actions)	Réalisée
Matching « Job idéal » (suggestion d'offres à partir d'un CV)	Réalisée
Entretien téléphonique RH obligatoire avant acceptation	Réalisée
Calendrier manager dédié (lecture seule, cloisonné par périmètre)	Réalisée
Retrait de candidature en libre-service par le candidat	Réalisée
Assistant conversationnel candidat (chatbot IA via OpenRouter, connaissance des candidatures réelles et de la plateforme)	Réalisée

4.8 Conclusion

Entre l'environnement de travail, les interfaces effectivement livrées, les tests menés et les difficultés qu'il a fallu résoudre en cours de route, ce chapitre reflète assez fidèlement le quotidien du développement de JOB4YOU. Reste maintenant à prendre du recul sur l'ensemble du projet — c'est l'objet de la conclusion générale qui suit.

Conclusion Générale et Perspectives

Sept mois après mon arrivée chez Sopra HR Software Tunisie pour ce Projet de Fin d'Études, JOB4YOU est passé d'un simple besoin identifié en tout début de stage à une plateforme fonctionnelle, capable de gérer le processus de recrutement interne de l'entreprise.

Ce travail couvre désormais l'ensemble du cycle de recrutement, de la publication d'une offre jusqu'à la décision finale d'embauche, en passant par la soumission des candidatures, l'analyse automatique des CV par intelligence artificielle, la planification des entretiens et la collecte des feedbacks des managers. La plateforme intègre aussi un système de notifications automatiques par e-mail et un tableau de pilotage visuel de type Kanban, qui donnent aux équipes RH et aux managers une visibilité en temps réel sur l'avancement de chaque dossier.

Sur le plan technique, ce projet m'a permis d'approfondir mes compétences en développement backend avec Spring Boot et Spring Security, en développement frontend avec Angular, ainsi qu'en conception de bases de données relationnelles avec PostgreSQL. J'y ai aussi découvert et mis en œuvre des outils d'automatisation (n8n) et d'intelligence artificielle (API Cohere) au service d'un processus métier concret, tout en renforçant mes compétences en sécurité applicative (authentification JWT, contrôle d'accès basé sur les rôles) et en gestion de projet, selon une démarche itérative inspirée d'Agile/Scrum.

Au-delà de l'aspect purement technique, cette expérience en alternance au sein d'un grand groupe international m'a beaucoup apporté sur le plan professionnel : travail en équipe, communication régulière avec mon encadrant entreprise, prise en compte de contraintes réelles de sécurité et de fiabilité, et nécessité de concevoir une solution robuste et évolutive dans un contexte professionnel réel.

Plusieurs pistes d'évolution restent ouvertes pour poursuivre ce travail. Côté fonctionnel, la plateforme pourrait accueillir un module d'analyse prédictive du taux de transformation des candidatures, une intégration plus poussée avec des plateformes d'emploi externes (LinkedIn, Indeed), ou un espace candidat plus riche pour un suivi encore plus détaillé de son parcours. Côté technique, des tests automatisés plus exhaustifs (unitaires et d'intégration), une chaîne d'intégration et de déploiement continus (CI/CD), ainsi qu'un déploiement sur une infrastructure cloud seraient des évolutions naturelles pour renforcer la fiabilité et la scalabilité de la plateforme sur le long terme.

Les objectifs fixés au départ sont atteints : JOB4YOU est une plateforme fonctionnelle et sécurisée, qui répond aux besoins identifiés et qui est prête à entrer en phase de préproduction avant son industrialisation, en vue d'un déploiement au sein de Sopra HR Software Tunisie. Au-delà du livrable, cette expérience m'a permis de consolider mes compétences techniques et méthodologiques dans un contexte professionnel réel, tout en développant ma capacité à concevoir, sécuriser, tester et faire évoluer une solution logicielle complète.

Bibliographie

- [1] Spring Boot Reference Documentation. Disponible sur : https://docs.spring.io/spring-boot/. Consulté le 03/08/2026.
- [2] Spring Security Reference Documentation. Disponible sur : https://docs.spring.io/spring-security/reference/. Consulté le 03/08/2026.
- [3] Angular Documentation. Disponible sur : https://angular.dev. Consulté le 03/08/2026.
- [4] PostgreSQL Documentation. Disponible sur : https://www.postgresql.org/docs/. Consulté le 03/08/2026.
- [5] JSON Web Token (JWT) — RFC 7519. Disponible sur : https://www.rfc-editor.org/rfc/rfc7519. Consulté le 03/08/2026.
- [6] Cohere API Documentation. Disponible sur : https://docs.cohere.com. Consulté le 03/08/2026.
- [7] n8n Documentation. Disponible sur : https://docs.n8n.io. Consulté le 03/08/2026.
- [8] OMG — Unified Modeling Language (UML) Specification. Disponible sur : https://www.omg.org/spec/UML. Consulté le 03/08/2026.
- [9] CNIL — Intelligence artificielle et RGPD, fiches pratiques. Disponible sur : https://www.cnil.fr/fr/intelligence-artificielle. Consulté le 03/08/2026.
- [10] Règlement (UE) 2016/679 (RGPD), article 22 — Décision individuelle automatisée. Disponible sur : https://eur-lex.europa.eu/eli/reg/2016/679/oj. Consulté le 03/08/2026.
- [11] OWASP Top 10 — Application Security Risks. Disponible sur : https://owasp.org/www-project-top-ten/. Consulté le 03/08/2026.
- [12] WALLS, Craig. Spring Boot in Action. Manning Publications, 2015.
- [13] FREEMAN, Adam. Pro Angular. 3e édition, Apress, 2019.
- [14] ISO 30415:2021 — Management des ressources humaines : Diversité et inclusion.

Glossaire

- API : Application Programming Interface — interface permettant à deux systèmes logiciels de communiquer.
- IA : Intelligence Artificielle.
- JPA : Java Persistence API — spécification Java pour la persistance des objets en base de données relationnelle.
- JWT : JSON Web Token — jeton signé utilisé pour authentifier les requêtes entre le frontend et le backend.
- MLD : Modèle Logique de Données.
- RBAC : Role-Based Access Control — contrôle d'accès basé sur les rôles des utilisateurs.
- REST : Representational State Transfer — style d'architecture pour les API web.
- UML : Unified Modeling Language — langage de modélisation graphique utilisé pour la conception logicielle.
- BCrypt : algorithme de hachage adaptatif utilisé pour le stockage sécurisé des mots de passe.
- CI/CD : Continuous Integration / Continuous Delivery, intégration et livraison continues.
- Cohere : fournisseur d'API d'intelligence artificielle utilisé pour l'analyse sémantique et le scoring des CV.
- Kanban : méthode de visualisation du flux de travail sous forme de colonnes représentant les étapes d'un processus.
- n8n : plateforme d'automatisation permettant de construire des workflows interconnectant différents services et API via des webhooks.
- ORM : Object-Relational Mapping, correspondance entre objets applicatifs et tables relationnelles (via JPA/Hibernate).
- Spring Security : module du framework Spring assurant l'authentification et le contrôle d'accès des applications Java.
- Webhook : mécanisme par lequel une application notifie automatiquement une autre via une requête HTTP déclenchée par un événement.